

UniKnect Gen1 Pro

测试指导

版本: 1.1

日期: 2026-04-02

状态: 临时文件



上海移远通信技术股份有限公司（以下简称“移远通信”）始终以为客户提供最及时、最全面的服务为宗旨。如需任何帮助，请随时联系我司上海总部，联系方式如下：

上海移远通信技术股份有限公司
上海市松江区泗泾镇外婆泾路 8 号 邮编：200233
电话：+86 21 5108 6236 邮箱：info@quectel.com

或联系我司当地办事处，详情请登录：<http://www.quectel.com/cn/support/sales.htm>。

如需技术支持或反馈我司技术文档中的问题，请随时登录网址：
<http://www.quectel.com/cn/support/technical.htm> 或发送邮件至：support@quectel.com。

前言

移远通信提供该文档内容以支持客户的产品设计。客户须按照文档中提供的规范、参数来设计产品。同时，您理解并同意，移远通信提供的参考设计仅作为示例。您同意在设计您目标产品时使用您独立的分析、评估和判断。在使用本文档所指导的任何硬软件或服务之前，请仔细阅读本声明。您在此承认并同意，尽管移远通信采取了商业范围内的合理努力来提供尽可能好的体验，但本文档和其所涉及服务是在“可用”基础上提供给您。移远通信可在未事先通知的情况下，自行决定随时增加、修改或重述本文档。

使用和披露限制

许可协议

除非移远通信特别授权，否则我司所提供硬软件、材料和文档的接收方须对接收的内容保密，不得将其用于除本项目的实施与开展以外的任何其他目的。

版权声明

移远通信产品和本协议项下的第三方产品可能包含受移远通信或第三方材料、硬软件和文档版权保护的相关资料。除非事先得到书面同意，否则您不得获取、使用、向第三方披露我司所提供的文档和信息，或对此类受版权保护的资料进行复制、转载、抄袭、出版、展示、翻译、分发、合并、修改，或创造其衍生作品。移远通信或第三方对受版权保护的资料拥有专有权，不授予或转让任何专利、版权、商标或服务商标权的许可。为避免歧义，除了正常的非独家、免版税的产品使用许可，任何形式的购买都不可被视为授予许可。对于任何违反保密义务、未经授权使用或以其他非法形式恶意使用所述文档和信息的违法侵权行为，移远通信有权追究法律责任。

商标

除另行规定，本文档中的任何内容均不授予在广告、宣传或其他方面使用移远通信或第三方的任何商标、商号及名称，或其缩略语，或其仿冒品的权利。

第三方权利

您理解本文档可能涉及一个或多个属于第三方的硬软件和文档（“第三方材料”）。您对此类第三方材料的使用应受本文档的所有限制和义务约束。

移远通信针对第三方材料不做任何明示或暗示的保证或陈述，包括但不限于任何暗示或法定的适销性或特定用途的适用性、平静受益权、系统集成、信息准确性以及与许可技术或被许可人使用许可技术相关的不侵犯任何第三方知识产权的保证。本协议中的任何内容都不构成移远通信对任何移远通信产品或任何其他软硬件、设备、工具、信息或产品的开发、增强、修改、分销、营销、销售、提供销售或以其他方式维持生产的陈述或保证。此外，移远通信免除因交易过程、使用或贸易而产生的任何和所有保证。

隐私声明

为实现移远通信产品功能，特定设备数据将会上传至移远通信或第三方服务器（包括运营商、芯片供应商或您指定的服务器）。移远通信严格遵守相关法律法规，仅为实现产品功能之目的或在适用法律允许的情况下保留、使用、披露或以其他方式处理相关数据。当您与第三方进行数据交互前，请自行了解其隐私保护和数据安全政策。

免责声明

- 1) 移远通信不承担任何因未能遵守有关操作或设计规范而造成损害的责任。
- 2) 移远通信不承担因本文档中的任何因不准确、遗漏、或使用本文档中的信息而产生的任何责任。
- 3) 移远通信尽力确保开发中功能的完整性、准确性、及时性，但不排除上述功能错误或遗漏的可能。除非另有协议规定，否则移远通信对开发中功能的使用不做任何暗示或法定的保证。在适用法律允许的最大范围内，移远通信不对任何因使用开发中功能而遭受的损害承担责任，无论此类损害是否可以预见。
- 4) 移远通信对第三方网站及第三方资源的信息、内容、广告、商业报价、产品、服务和材料的可访问性、安全性、准确性、可用性、合法性和完整性不承担任何法律责任。

版权所有 ©上海移远通信技术股份有限公司 2025，保留一切权利。

Copyright © Quectel Wireless Solutions Co., Ltd. 2023.

文档历史

修订记录

版本	日期	作者	变更描述
1.0	2026-03-10	Mandy.Wang	初始版本
1.1	2026-04-10	Mandy.Wang	更新工具图片与优化一些描述

目录

文档历史	3
目录	4
图片索引	5
表格索引	7
1 介绍	8
2 环境准备	9
2.1. 组件介绍	9
2.1.1. UniKnect Gen1-PRO 扩展板	10
2.1.2. STM32 NUCLEO-F413ZH 开发板	11
2.1.3. LCD 扩展开发板	11
2.1.4. 组装开发板	12
2.1.5. 安装驱动, 烧录固件, 开发环境	12
2.2. 简单操作流程	12
2.2.1. 开机流程	12
2.2.2. 端口说明	13
3 功能测试	14
3.1. 移远物联网应用协议在线测试平台	14
3.2. 使用 Thonny 工具连接设备	16
3.3. 温湿度传感器测试	17
3.4. 加速度传感器测试	18
3.5. LCD 测试	19
3.6. I2C 测试	22
3.7. ADC 测试	23
3.8. 线程测试	24
3.9. GNSS 测试	25
3.10. TCP 客户端测试	26
3.11. HTTP 客户端测试	29
3.12. MQTTS 客户端测试	33
3.13. FTP 客户端测试	35
4 参考文档	37

图片索引

图 1: 配件组装.....	9
图 2: UniKnect Gen1-PRO 扩展板.....	10
图 3: NUCLEO-F413ZH 开发板.....	11
图 4: LCD 扩展开发板.....	12
图 5: 安装 4G 模组驱动与调试器驱动.....	13
图 6: Github 资源.....	14
图 7: Quectel 物联网应用协议在线测试平台 ConnectLab.....	15
图 8: ConnectLab TCP Server 创建.....	15
图 9: ConnectLab TCP Server 配置.....	16
图 10: Thonny 界面配置.....	17
图 11: 读取温度/湿度传感器数据.....	17
图 12: 停止打印温度/湿度数据.....	18
图 13: 读取加速度传感器数据.....	18
图 14: images.py 与 images1.py 数据上传至 flash.....	19
图 15: images.py 与 images1.py 成功上传至 flash.....	20
图 16: 运行 LCD 程序.....	21
图 17: LCD 扩展开发板.....	21
图 18: 运行 LCD1 程序.....	22
图 19: LCD 扩展开发板.....	22
图 20: I2C 测试.....	23
图 21: ADC 测试.....	24
图 22: 线程测试.....	25
图 23: 读取定位经纬度数据.....	26
图 24: TCP 服务器地址与端口配置.....	27
图 25: 建立 TCP 连接.....	28
图 26: 发送 TCP 数据.....	28
图 27: ConnectLab TCP 服务器接收数据.....	28
图 28: 关闭 TCP 连接.....	29
图 29: ConnectLab TCP 服务器运行结果.....	29
图 30: ConnectLab HTTP 服务器.....	30
图 31: ConnectLab HTTP 服务器配置.....	30
图 32: HTTP GET 配置.....	31
图 33: HTTP POST 配置.....	31
图 34: HTTP GET 运行结果.....	32
图 35: HTTP POST 运行结果.....	32
图 36: ConnectLab HTTP 服务器运行结果.....	32
图 37: ConnectLab MQTTS 服务器.....	33
图 38: ConnectLab MQTTS 服务器证书下载.....	33
图 39: MQTTS 证书上传至 flash.....	34
图 40: MQTTS 配置及运行结果.....	35
图 41: ConnectLab MQTTS 服务器运行结果.....	35

图 42: ConnectLab FTP 服务器运行结果	36
------------------------------------	----

Quectel Confidential

表格索引

表 1: 组件列表.....	9
表 2: 参考文档.....	37

Quectel Confidential

1 介绍

本文档主要介绍 Quectel UniKnect Gen1 Pro 项目的测试指导，主要包括环境准备、软件布置、简单操作流程、测试指导等。

Quectel Confidential

2 环境准备

在测试之前，需要准备好硬件环境，驱动安装，烧录固件，开发工具。

2.1. 组件介绍

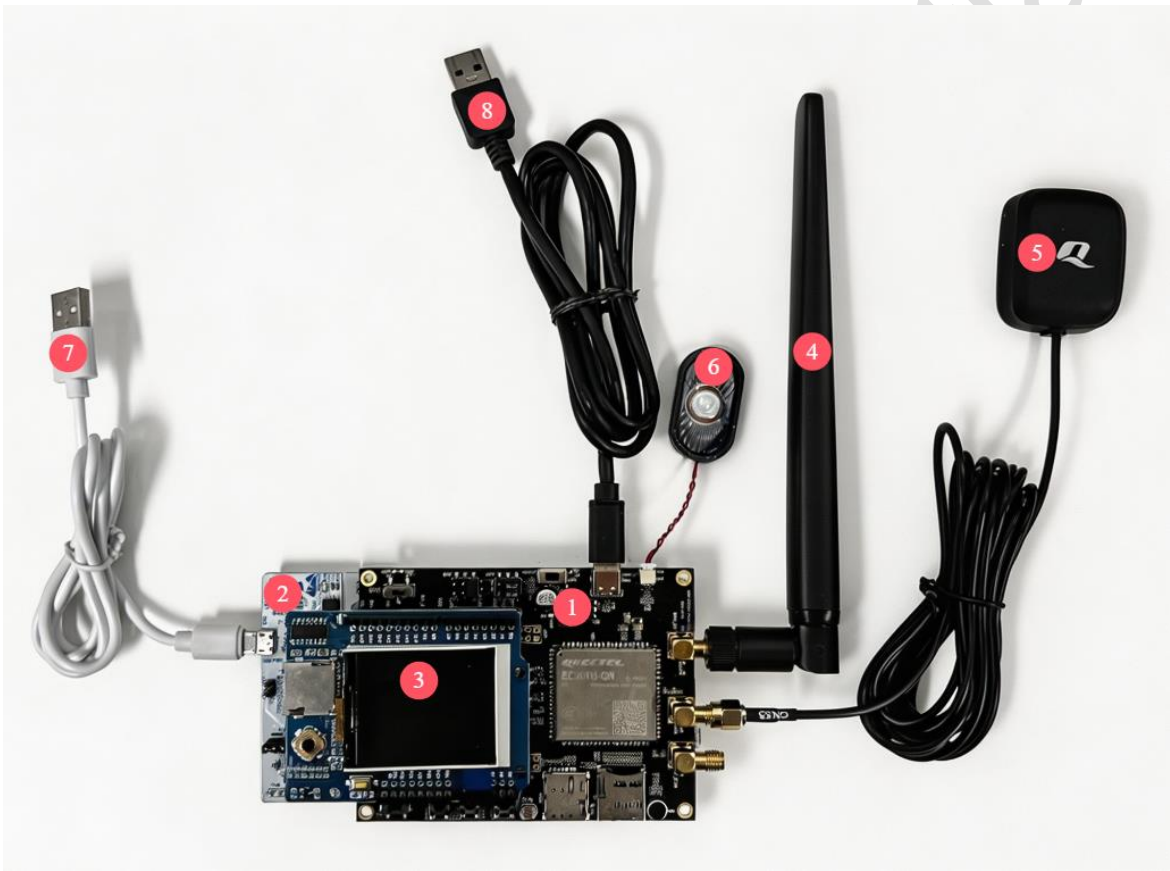


图 1: 配件组装

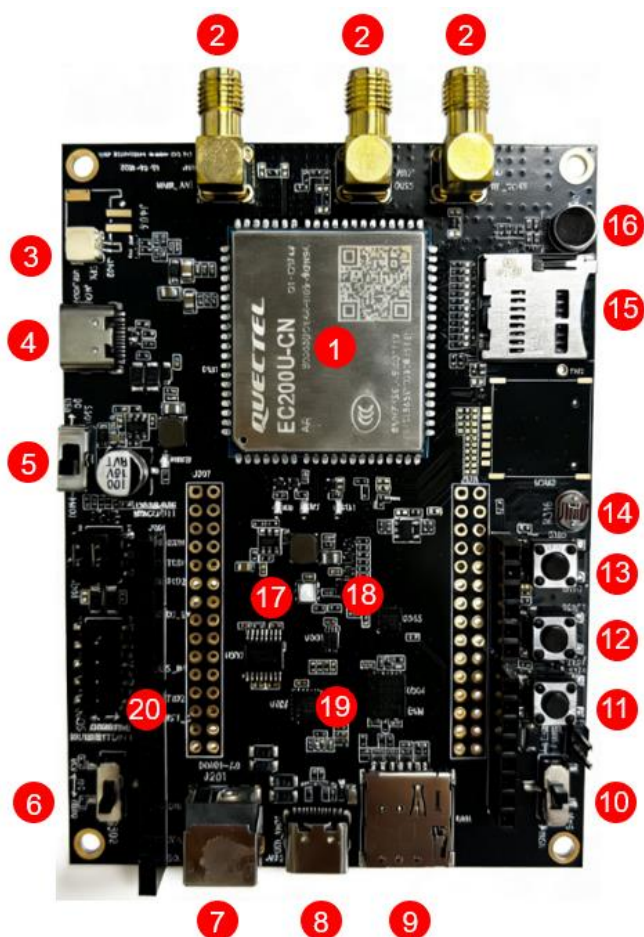
表 1: 组件列表

序号	组件
1	UniKnect Gen1-PRO 扩展板
2	NUCLEO-F413ZH 开发板

3	LCD 扩展开发板
4	LTE 蜂窝天线，用于蜂窝模组信号接收和发送
5	GNSS 天线，用于定位
6	喇叭，用于语音外放
7	Micro USB 线，用于 STM32 固件烧录与功能测试
8	Type-c USB 线，用于给蜂窝模组供电，模组版本升级或者 AT 命令操作

2.1.1. UniKnect Gen1-PRO 扩展板

UniKnect Gen1-PRO 扩展板（OC: QADP-EC200UCNAA-Ardunio-EVB），是移远专门为物联网项目学习、开发用途定制。



- ① EC200U 4G 无线通信模组
- ② 天线
- ③ 扬声器接口
- ④ 4G 模组 USB 接口
- ⑤ 电源开关
- ⑥ I2C 设备连接模组/MCU 切换
- ⑦ DC5V 电源输入
- ⑧ 4G 模组 debug 调试串口
- ⑨ NanoSIM 插拔卡接口
- ⑩ ESIM 卡/插拔 SIM 卡切换
- ⑪ 4G 模组 PWRKEY 键（开关机）
- ⑫ 4G 模组 RESET 键（复位）
- ⑬ 4G 模组 USB_BOOT 键（强制下载）
- ⑭ 光敏电阻
- ⑮ TF 存储卡接口
- ⑯ 麦克风
- ⑰ 温湿度传感器
- ⑱ G-Sensor 传感器
- ⑲ ESIM 卡
- ⑳ Arduino UNO 标准接口

图 2: UniKnect Gen1-PRO 扩展板

2.1.2. STM32 NUCLEO-F413ZH 开发板

NUCLEO-F413ZH 开发板的核心角色是运行 MicroPython 系统及用户程序、并通过 Arduino UNO 接口与 UniKnect Gen1-PRO 扩展板连接，从而驱动 4G 模组实现物联网功能。

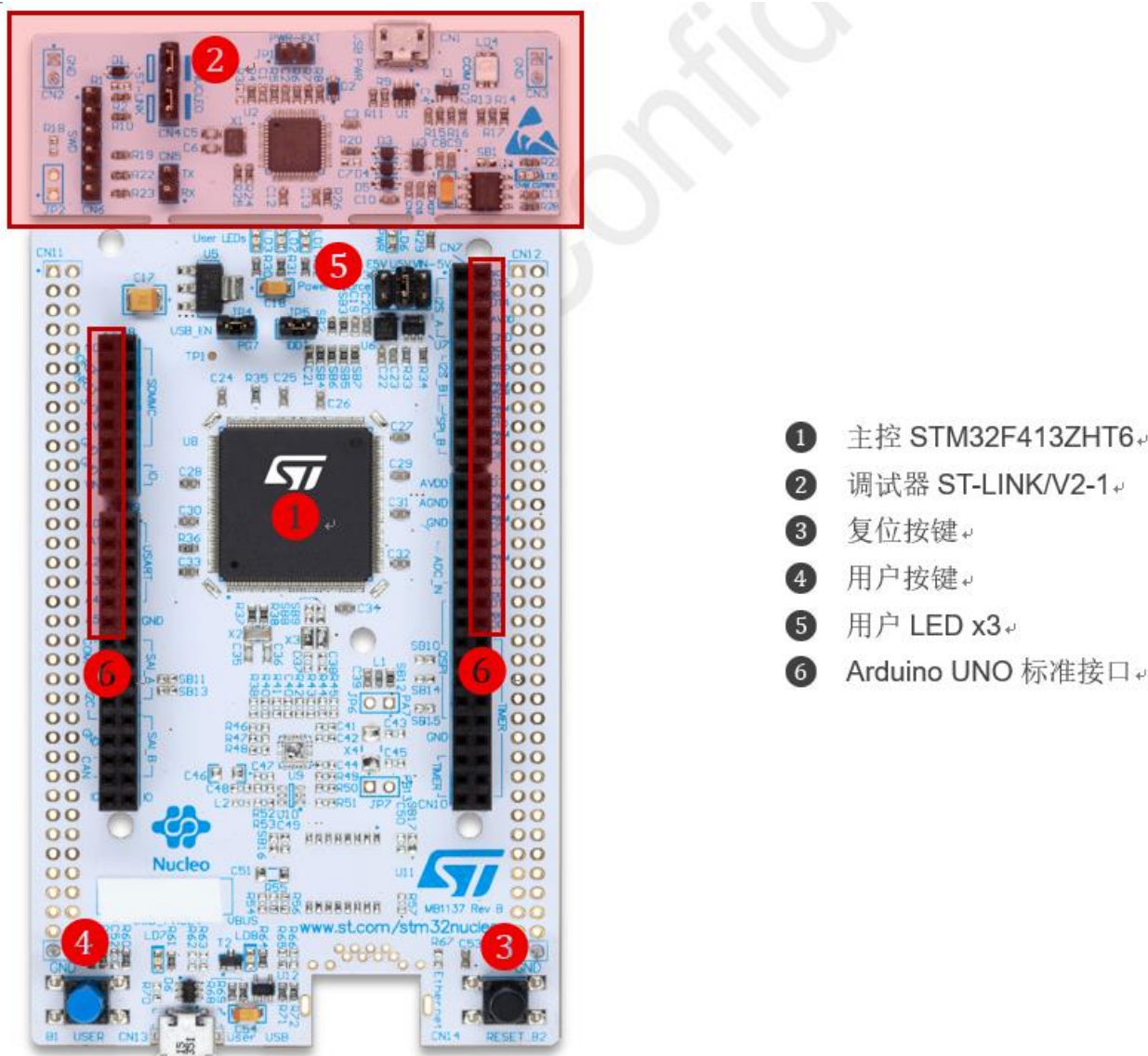


图 3: NUCLEO-F413ZH 开发板

2.1.3. LCD 扩展开发板

得益于 UniKnect Gen1-PRO 板载 Arduino UNO 标准接口，开发者可接入丰富的 Arduino 生态 shields 扩展板。或者根据项目功能需要，通过 NUCLEO-F413ZH 开发板引出的 MCU 引脚，接入其他硬件外设，实现项目整体功能。

如下图所示，在 UniKnect Gen1-PRO 扩展板上又垂直堆叠了一块 1.8 寸 LCD，用于交互显示。

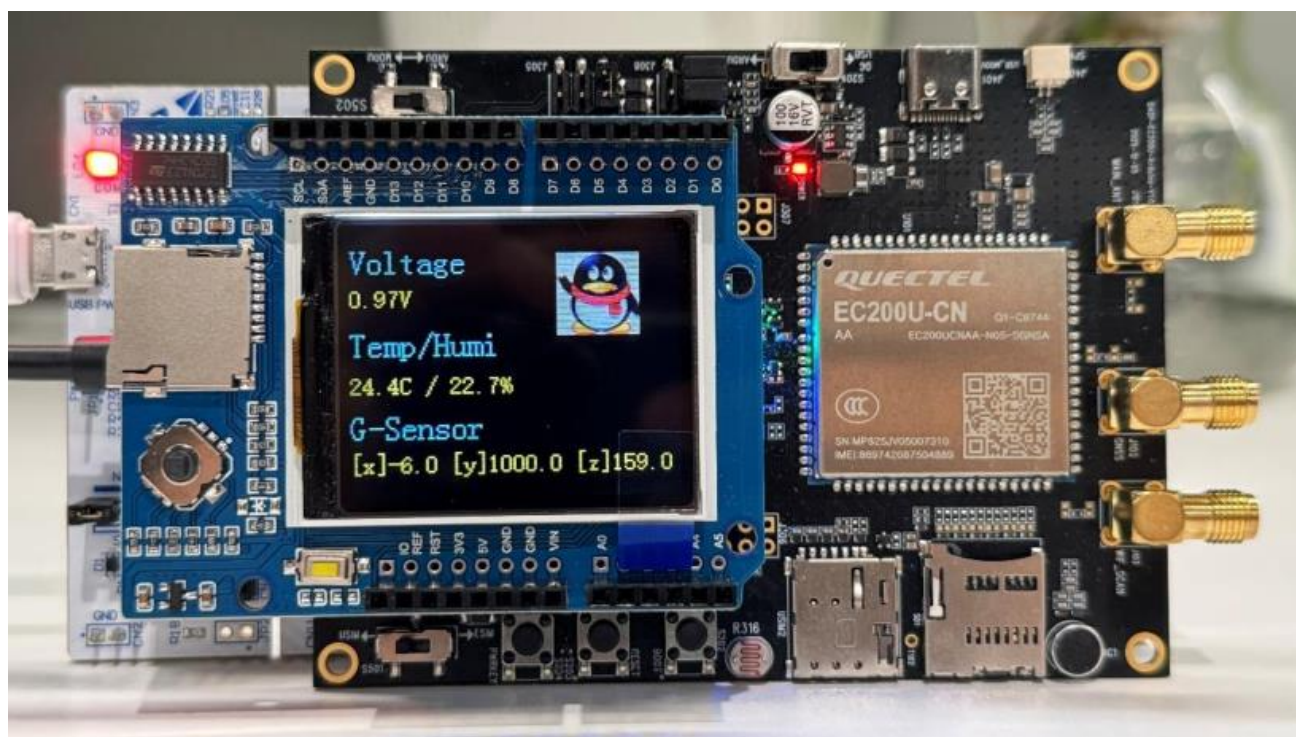


图 4: LCD 扩展开发板

2.1.4. 组装开发板

UniKnect Gen1-PRO 扩展板与 NUCLEO-F413ZH 开发板的物理连接非常简洁,采用垂直堆叠插接的方式。这种方式将两块板子组合为一个紧凑的整体,如同拼搭积木一般,两者间的 UART、GPIO、电源等关键信号也通过 Arduino UNO 接口自动连通,无需额外飞线,保证了硬件电路的简洁、稳定、可靠。详细请见参考文档[2]

2.1.5. 安装驱动, 烧录固件, 开发环境

详细请见参考文档[2]

2.2. 简单操作流程

2.2.1. 开机流程

1. 如 2.1.4 章节, 组装好开发板, 如图 1-序号 1、2、3。
2. UniKnect Gen1-PRO 扩展板内置贴片物联网卡, 可以直接使用。或选择 USIM1(Nano SIM 卡座)上插入实网卡。通过板子上的 S501(SIM 卡选择开关)来切换物联网卡与实网卡, 如图 2-序号 11
3. 连接 LTE 天线与 GNSS 天线, 如图 1-序号 4、5。

4. 无线蜂窝模组的电源供电有 3 种方式，J201，J401，J403 都可以作为电源输入口。S201 电源开关需要拨动到丝印 DC、USB 侧。我们选择其中一种方式：USB 接口（J401）连接电脑或者 USB（5V），如图 1-序号 8。
5. 使用 Micro USB 线连接 STM32 NUCLEO-F413ZH 开发板的与电脑，使用此口可以升级烧录 MCU 固件与功能测试，如图 1-序号 7。
6. 长按 UniKnect Gen1-PRO 扩展板上的 PWRKEY 按钮 2 秒以上再释放，如图 2-序号 11，之后蜂窝模组可以成功开机，查看板上 STAT 灯常亮。
7. 待 4G 蜂窝模组 EC200U-CN 成功注册上 4G 网络，UniKnect Gen1-PRO 扩展板上的 NET 灯会常亮，MOD 灯会闪烁。

2.2.2. 端口说明

使用 Micro USB 线连接 STM32 NUCLEO-F413ZH 开发板与电脑，使用此口进行功能测试。如图 1-序号 7。

使用 Type-c USB 线连接 STM32 NUCLEO-F413ZH 开发板的 J401 与电脑，如图 1-序号 8。使用此端口与模组进行通信。

安装对应驱动后，在设备管理器里可以查看如下图端口。

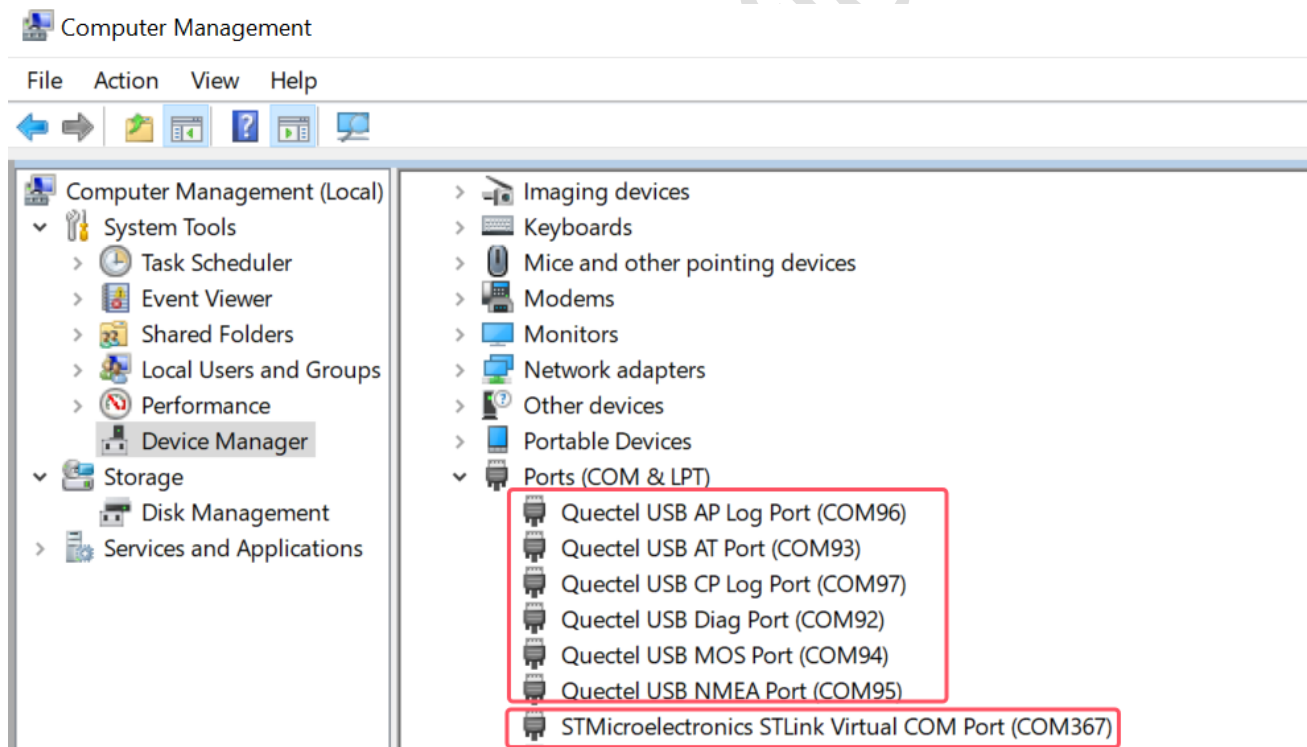


图 5: 安装 4G 模组驱动与调试器驱动

3 功能测试

从 Github 链接 <https://github.com/quectel-develop/micropython/tree/master/> 下载说明文档与 example 代码。基于 example 代码进行测试，或者进行代码修改，完善功能安排测试。

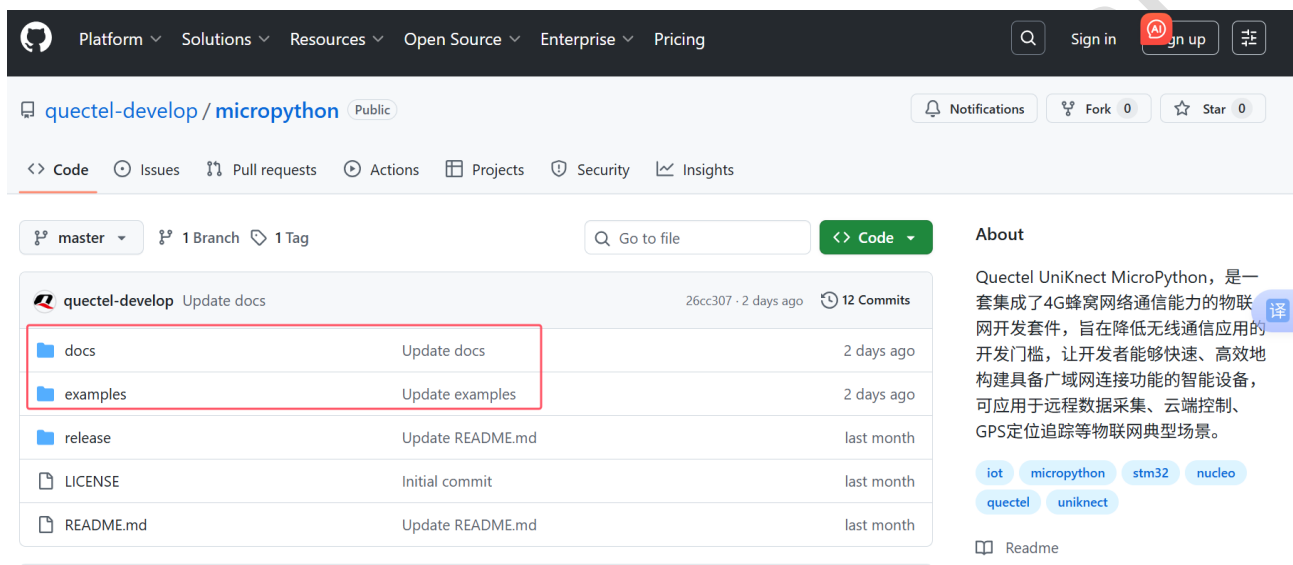


图 6: Github 资源

3.1. 移远物联网应用协议在线测试平台

Quectel 有自己的物联网应用协议在线测试平台。当前已经支持 TCP/UDP/SSL/MQTT(S)/HTTP 服务器。

登录网址: <https://connectlab.phicotek.com>, 选择游客模式。



图 7: Quectel 物联网应用协议在线测试平台 ConnectLab

如果需要打开 TCP 服务器，请选择 TCP->新建->IPV4，之后即可查看 TCP 服务器地址与端口。

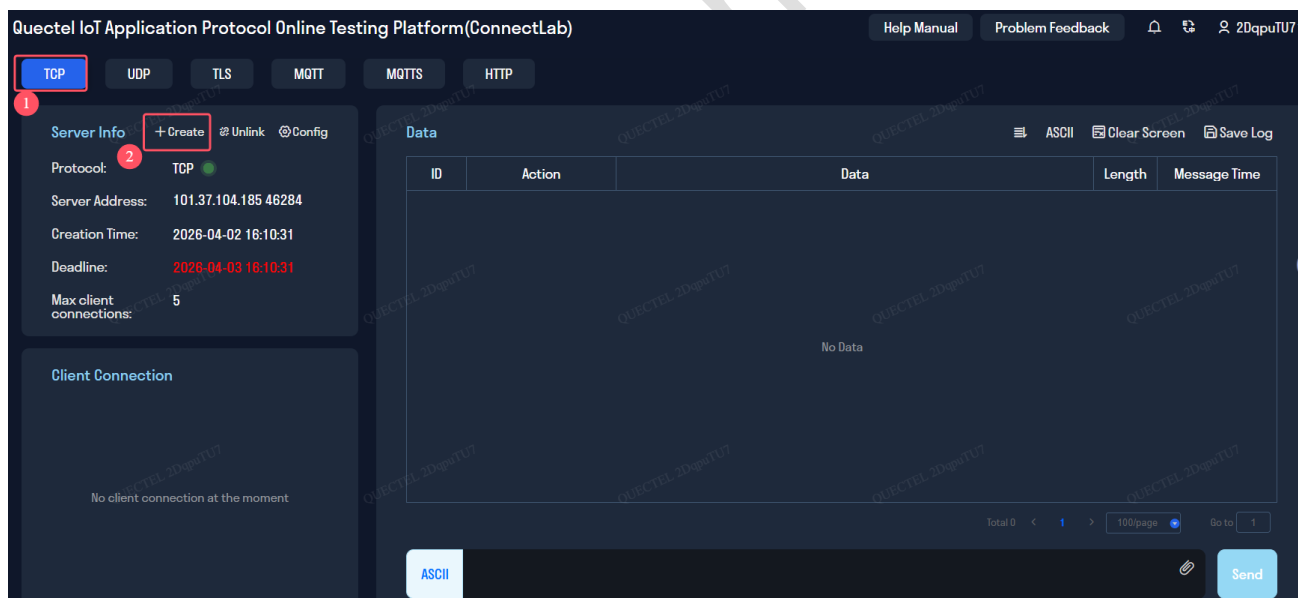


图 8: ConnectLab TCP Server 创建

支持自动发送与回复配置，比如配置自动回复收到的数据。

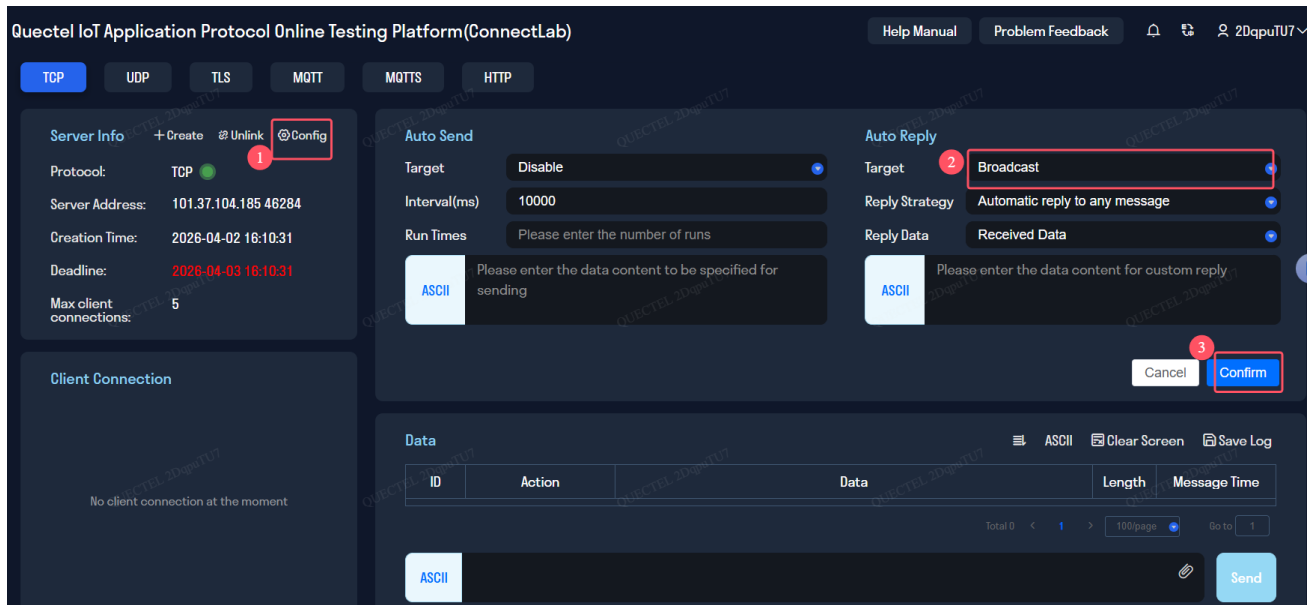


图 9: ConnectLab TCP Server 配置

3.2. 使用 Thonny 工具连接设备

参考文档[2]安装 Thonny 工具。如下图所示,选择端口 STLink Virtual COM Port 进行连接,打开 example 文件夹里的不同功能的测试代码,修改配置,点击运行。

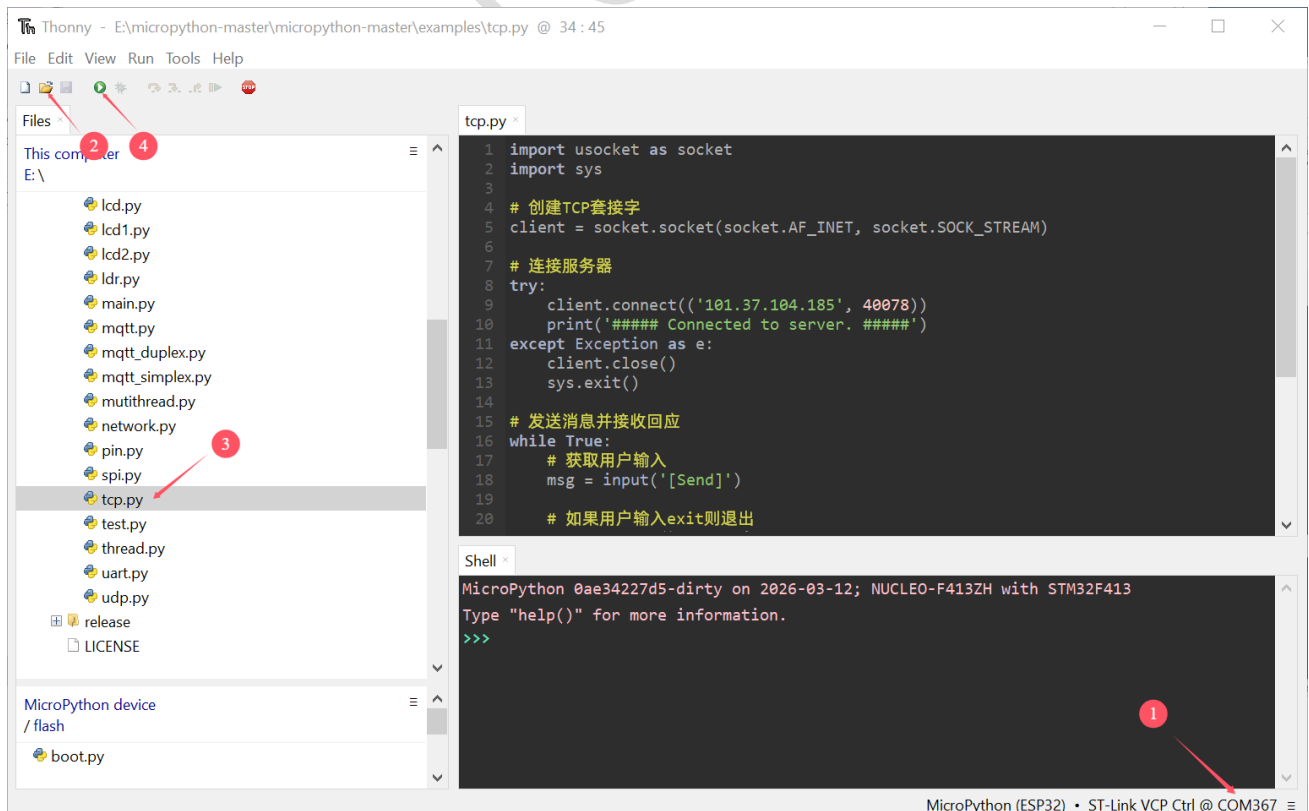


图 10: Thonny 界面配置

界面配置说明：

- 1: 用于连接设备
- 2: 打开 example 代码
- 3: 双击选中需要执行的代码
- 4: 点击运行

3.3. 温湿度传感器测试

aht2p.py 用于读取温湿度传感器的数据，运行结果如下图所示。Ctrl+C 用于停止打印温湿度数据。

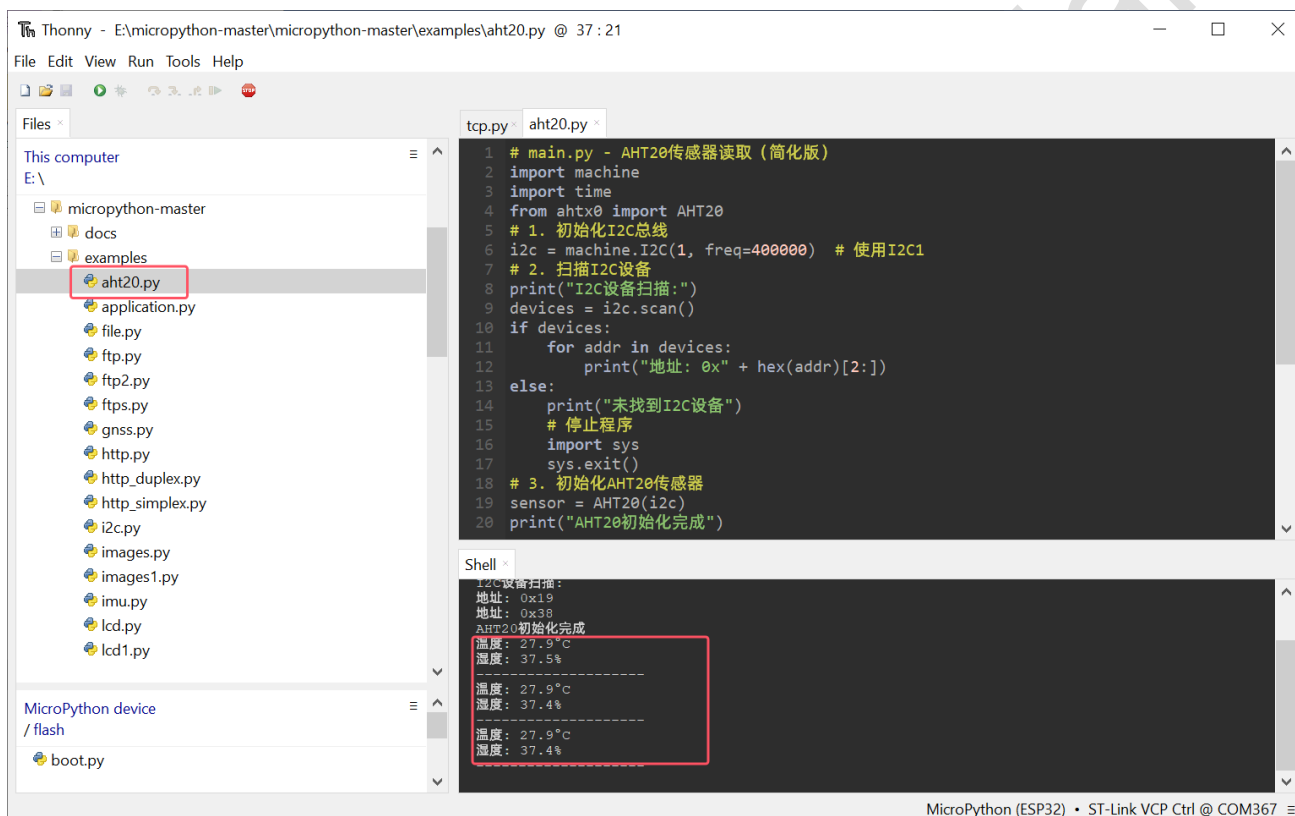


图 11: 读取温度/湿度传感器数据



图 12: 停止打印温度/湿度数据

3.4. 加速度传感器测试

imu.py 用于加速度传感器测试，请查看下图运行结果。

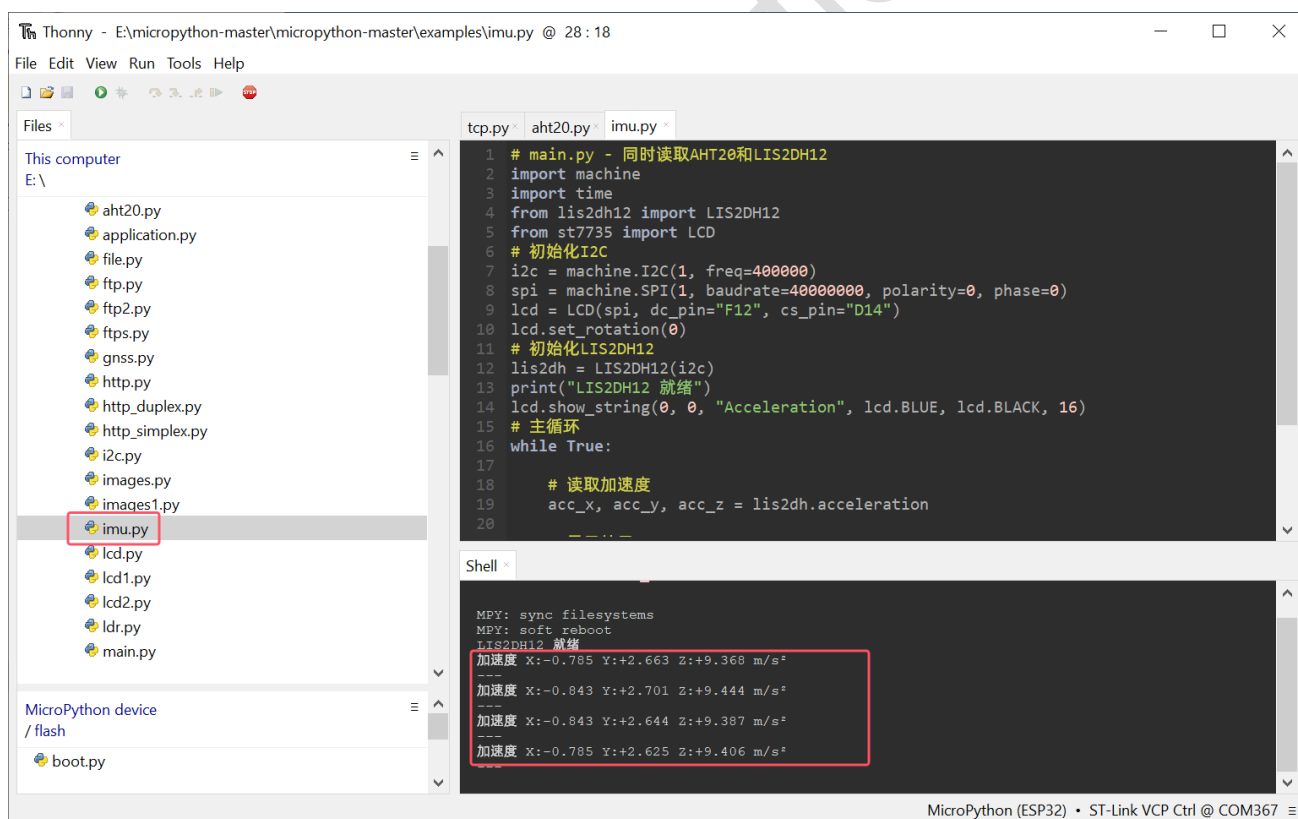


图 13: 读取加速度传感器数据

3.5. LCD 测试

lcd.py 用于 LCD 显示屏测试。

需要把 example 文件夹里的 images.py 与 images1.py 上传至 flash。具体方法：在 Thonny 工具上，右击 images.py，选择“上传到/flash”。相同的操作，用于 images1.py。

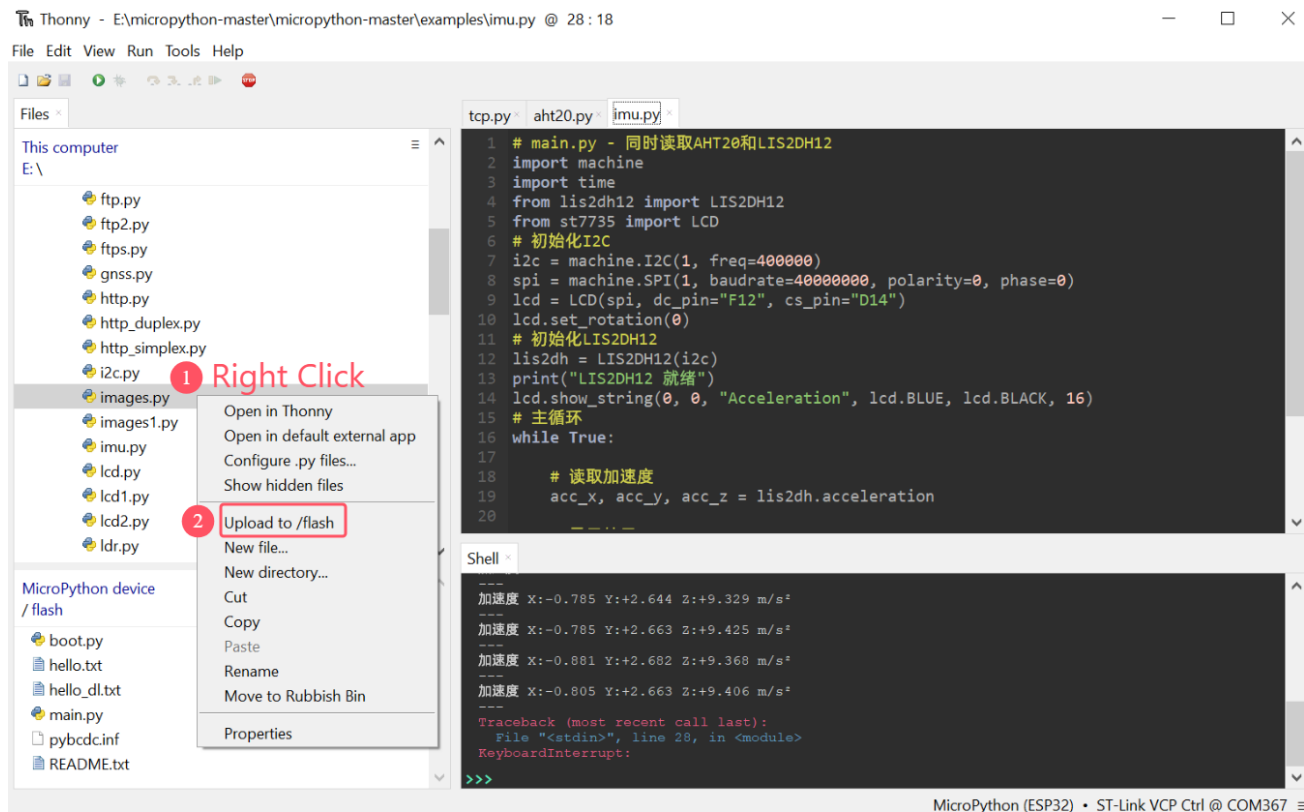


图 14: images.py 与 images1.py 数据上传至 flash

之后确认，flash 里有 images.py 与 images1.py 2 个文件。

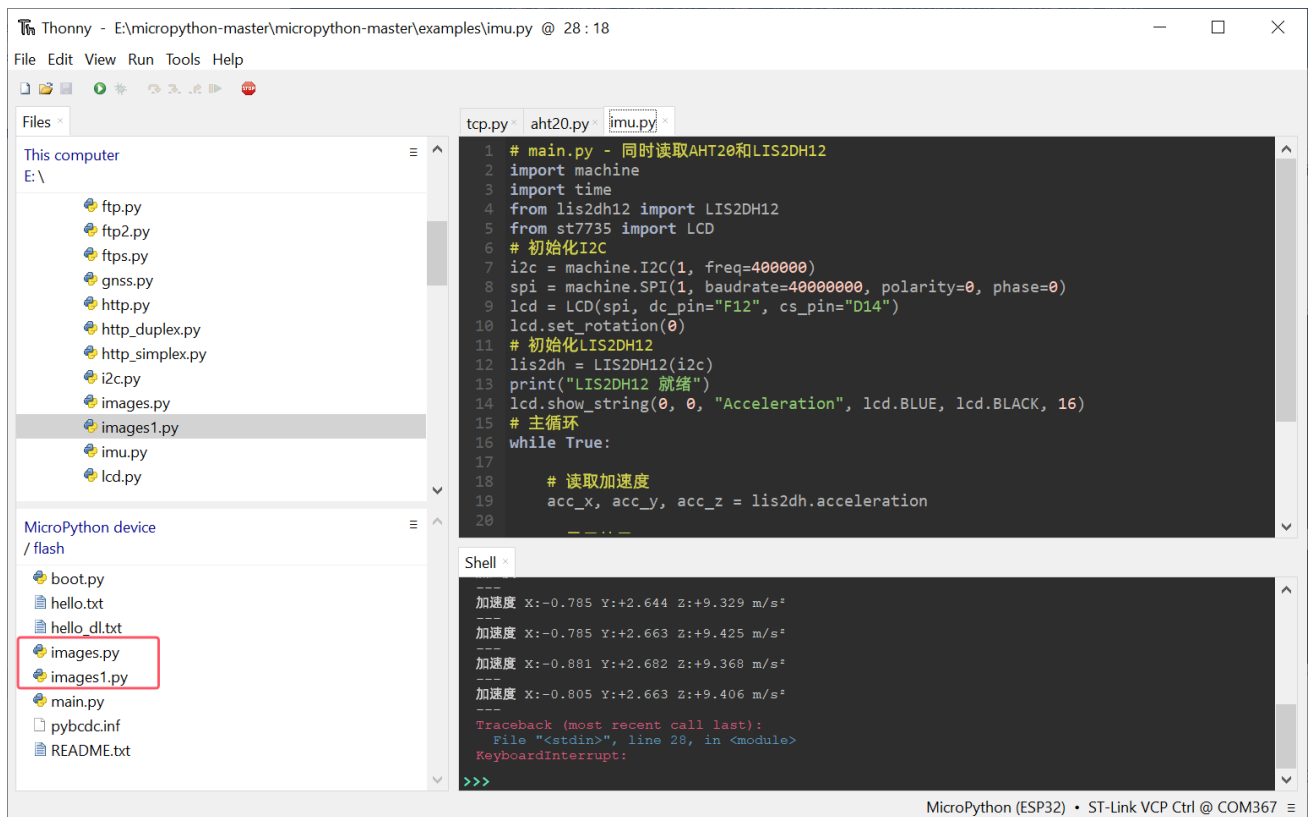


图 15: images.py 与 images1.py 成功上传至 flash

点击 Thonny 上的运行按钮来运行代码。之后，LCD 显示屏幕上显示 QUECTEL 图片。

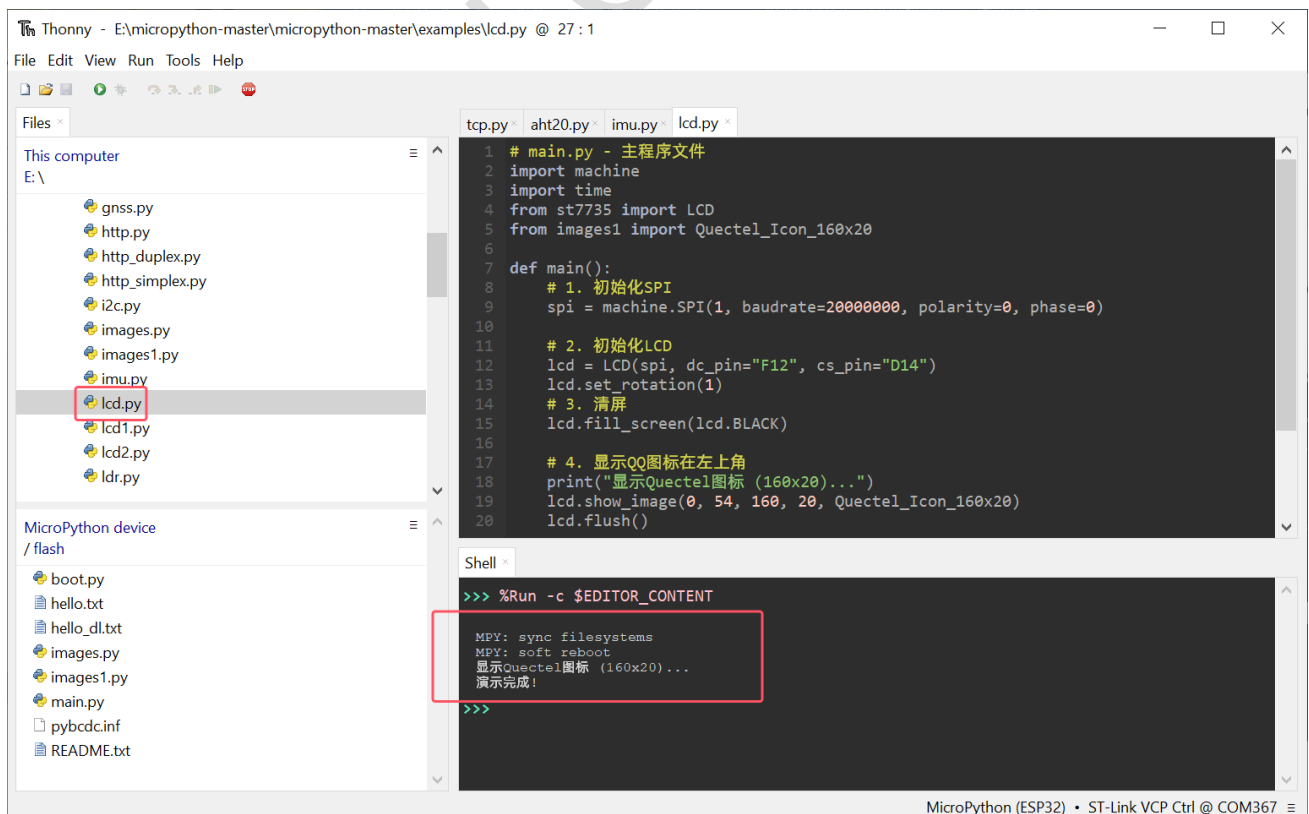


图 16: 运行 LCD 程序

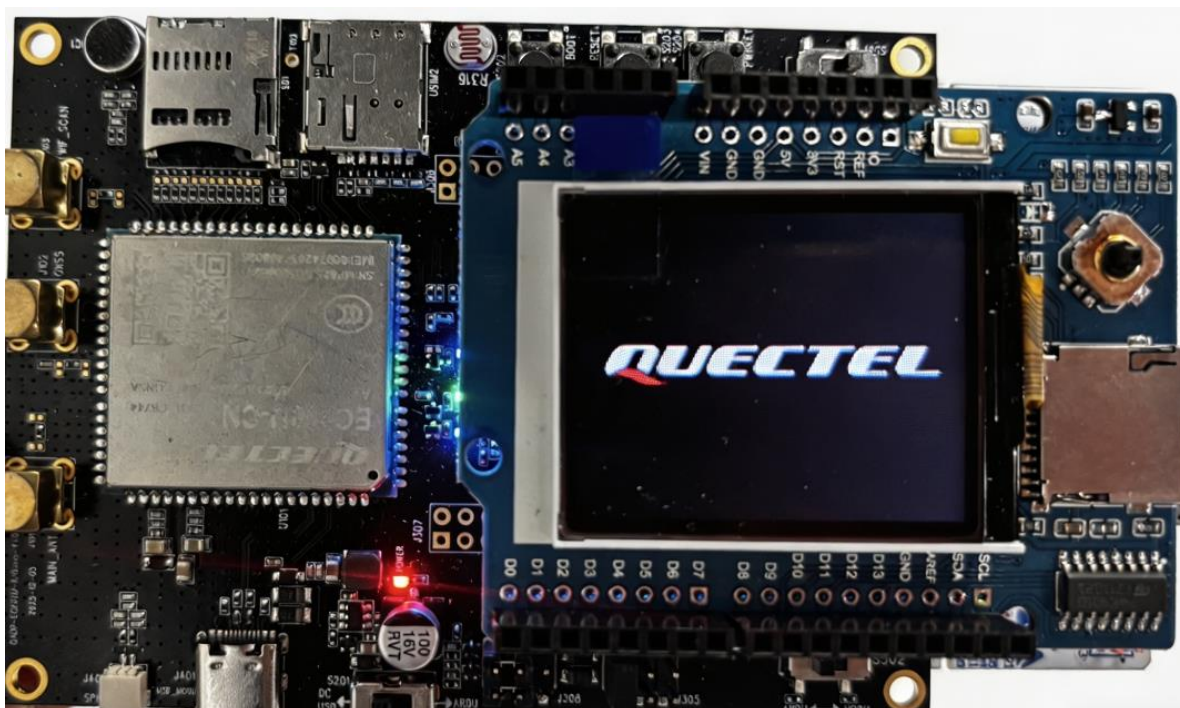


图 17: LCD 扩展开发板

lcd1.py 用于显示电压/温度/湿度 3 个数据。运行结果如下图所示：

```

1 import machine
2 import time
3 import random
4 from st7735 import LCD
5 # 初始化SPI
6 spi = machine.SPI(1, baudrate=4000000, polarity=0, phase=0)
7 lcd = LCD(spi, dc_pin="F12", cs_pin="D14")
8 lcd.set_rotation(1)
9 # 模拟传感器数据
10 def get_sensor_data():
11     # 模拟ADC电压 (0-3.3V)
12     voltage = random.uniform(3.0, 3.5)
13     # 模拟温度湿度
14     temp = random.uniform(20.0, 30.0)
15     humi = random.uniform(40.0, 60.0)
16     return voltage, temp, humi
17
18 # 主显示函数
19 def sensor_display():
20     # 清屏

```

```

MPY: sync filesystems
MPY: soft reboot
Sensor test start.
---## Voltage:      3.50V
---## Temp/Humi:    21.4C / 44.7%
---## Voltage:      3.40V
---## Temp/Humi:    28.1C / 46.2%
---## Voltage:      3.24V
---## Temp/Humi:    21.9C / 47.2%
---## Voltage:      3.28V
---## Temp/Humi:    20.1C / 54.6%

```


图 18: 运行 LCD1 程序

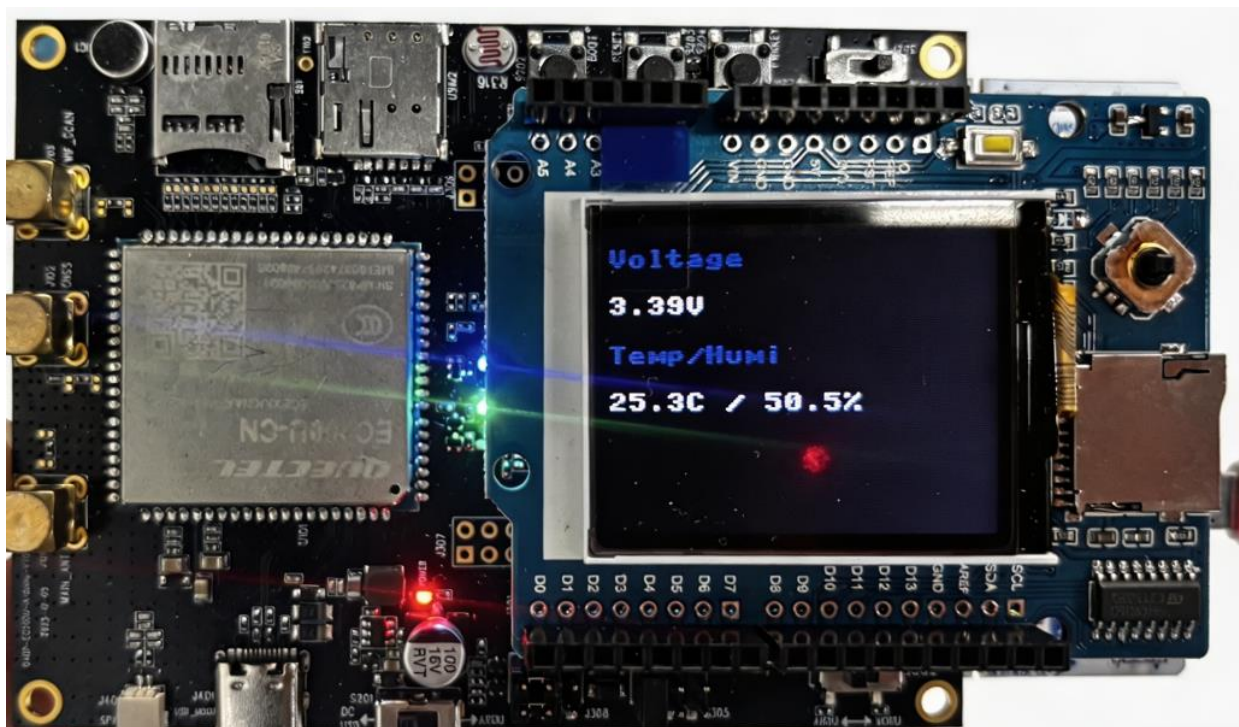


图 19: LCD 扩展开发板

3.6. I2C 测试

i2c.py 用于 I2C 测试。如下图查看运行结果。

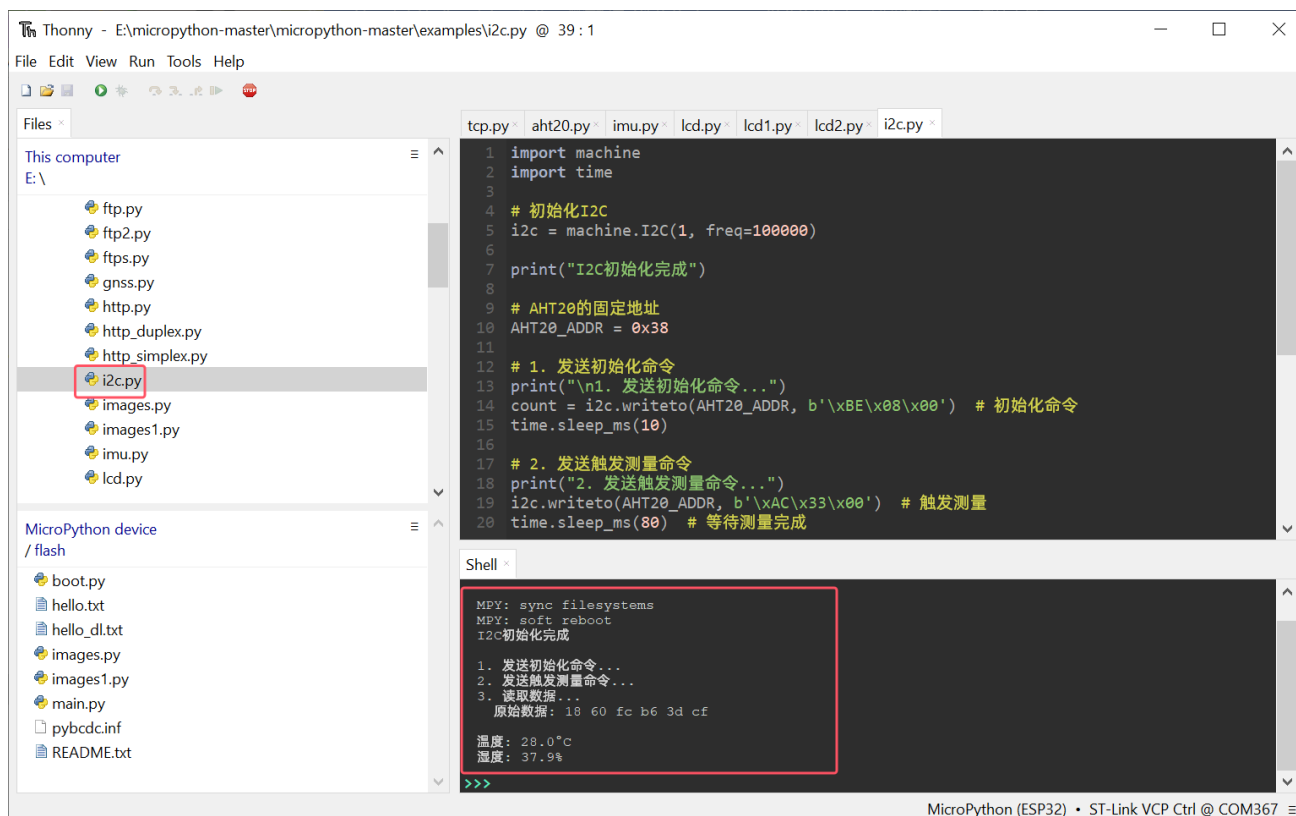


图 20: I2C 测试

3.7. ADC 测试

ldr.py 用于 ADC 测试。以板载自带光敏电阻为例，连接到 Arduino 接口的 ADC 引脚，MCU 可采集外部环境亮度。如下图查看运行结果。CTRL+C 用于停止运行。

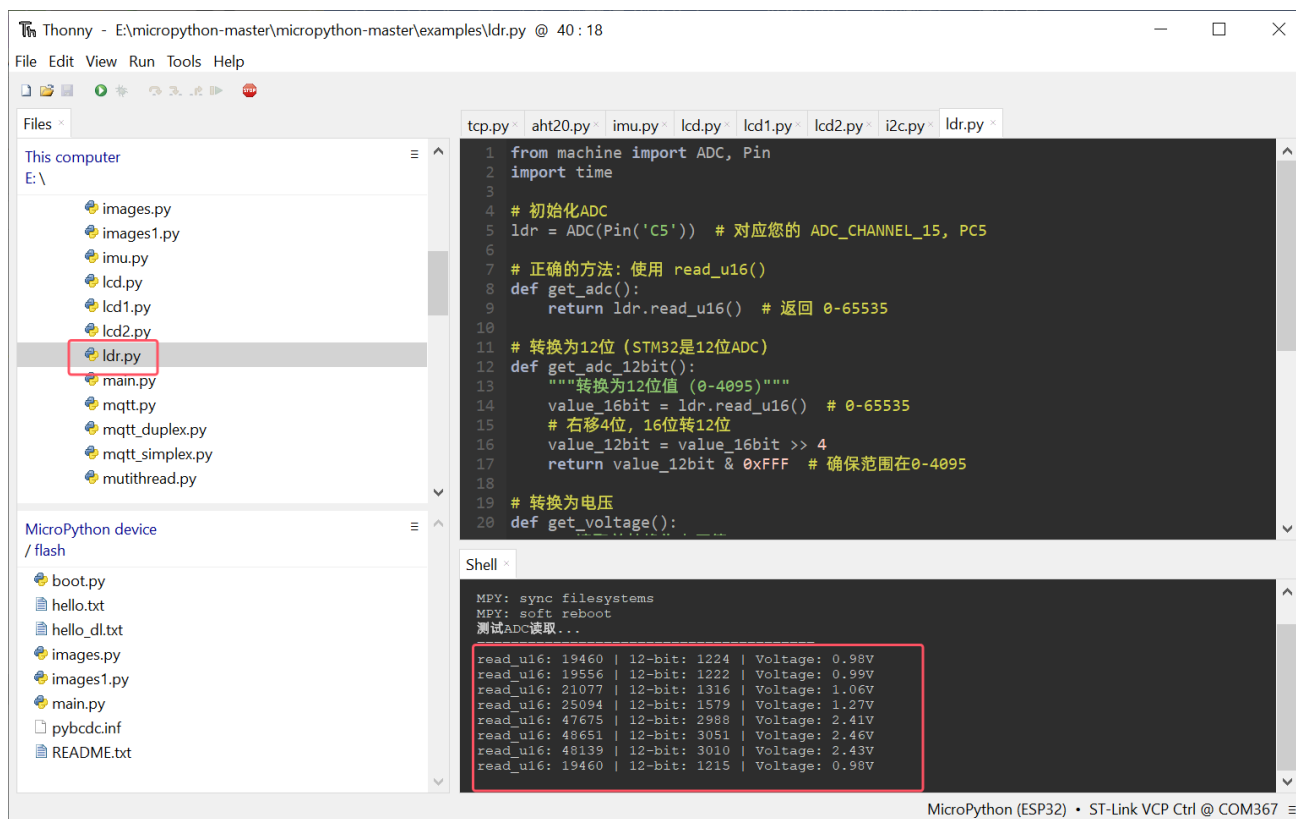


图 21: ADC 测试

3.8. 线程测试

Thread.py 用于线程相关测试

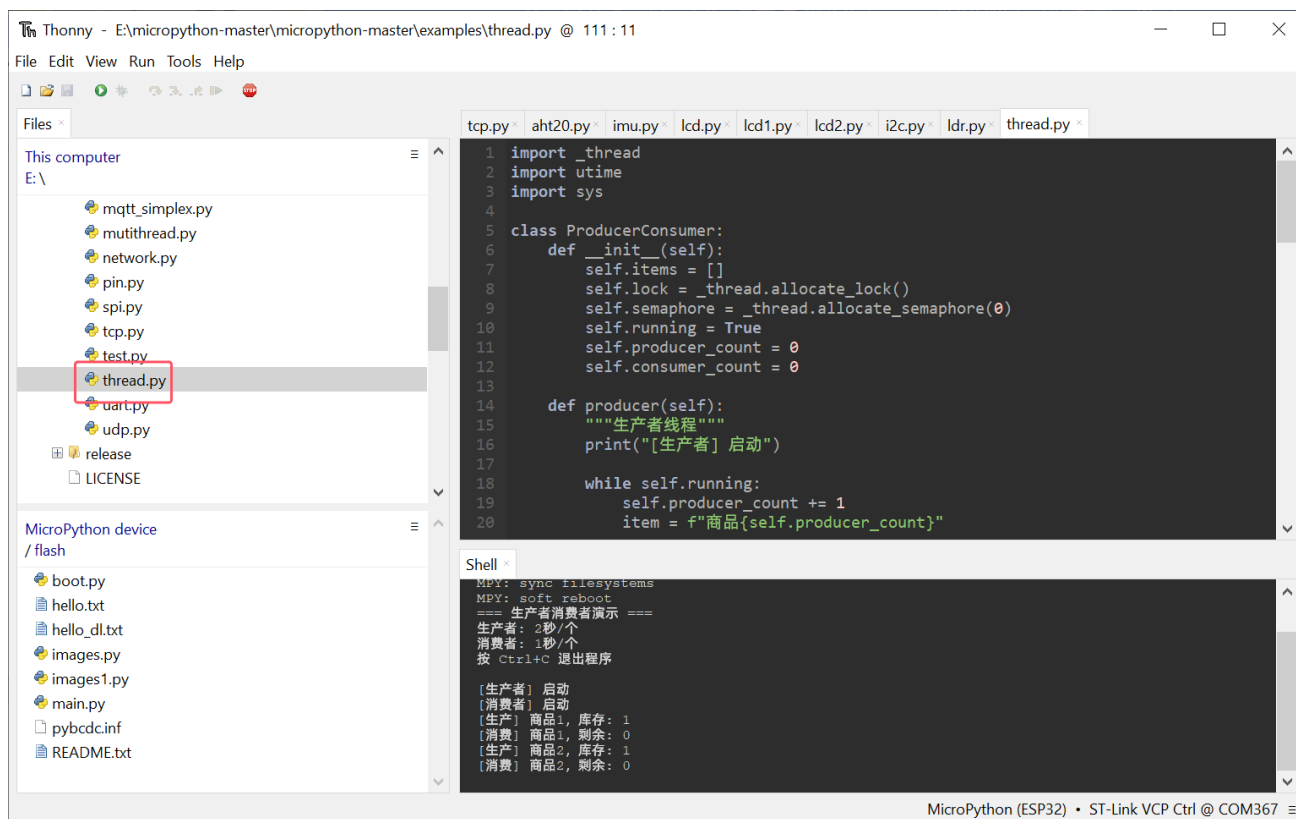


图 22: 线程测试

3.9. GNSS 测试

gnss.py 用于 GNSS 定位功能测试，请查看下图运行结果。CTRL+C 用于终止定位。
注意：需要把 GNSS 天线置于室外。第一次成功定位耗时较长，半分钟左右。

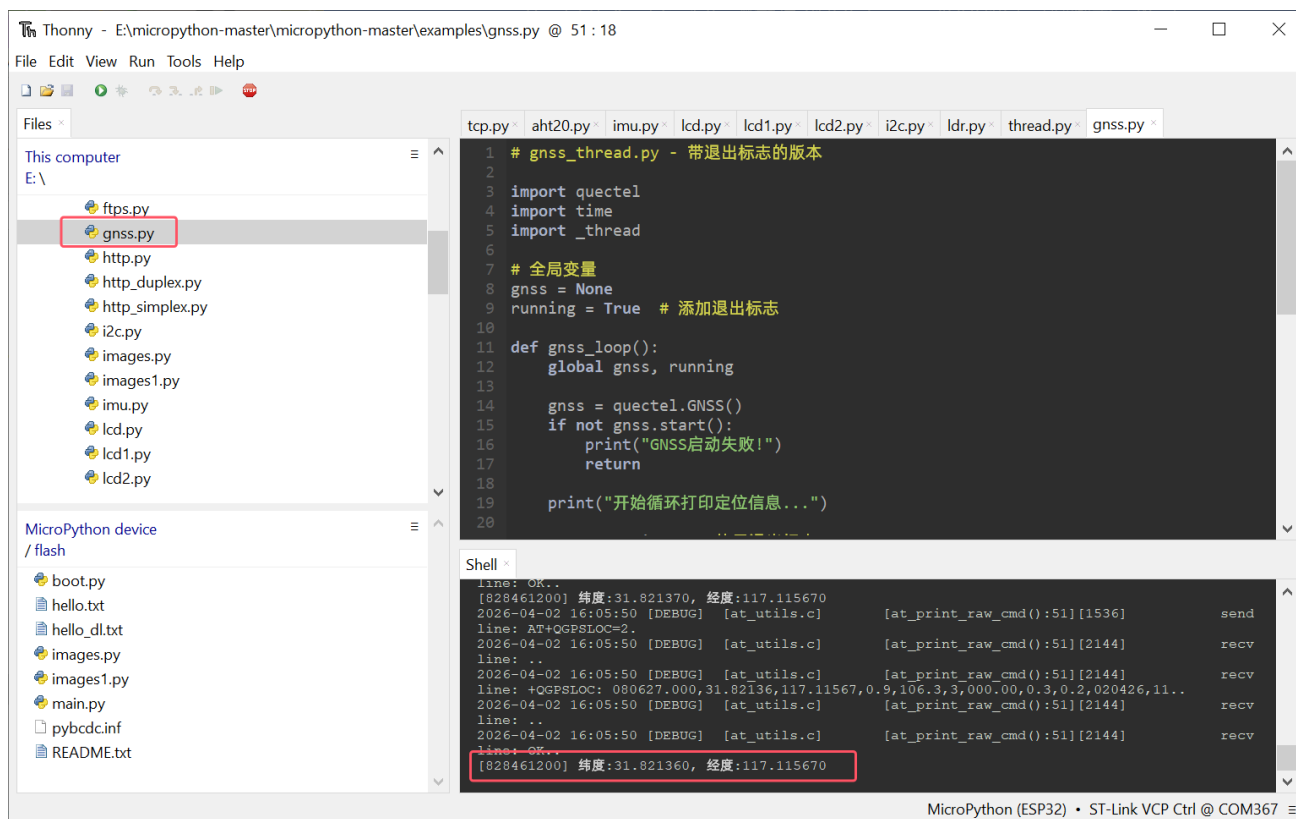


图 23: 读取定位经纬度数据

3.10. TCP 客户端测试

tcp.py 用于 TCP/UDP 测试。打开 tcp.py，如下图所示，修改 TCP 服务器地址与端口，可以配置使用 移远物联网应用协议在线测试平台 打开的服务器。

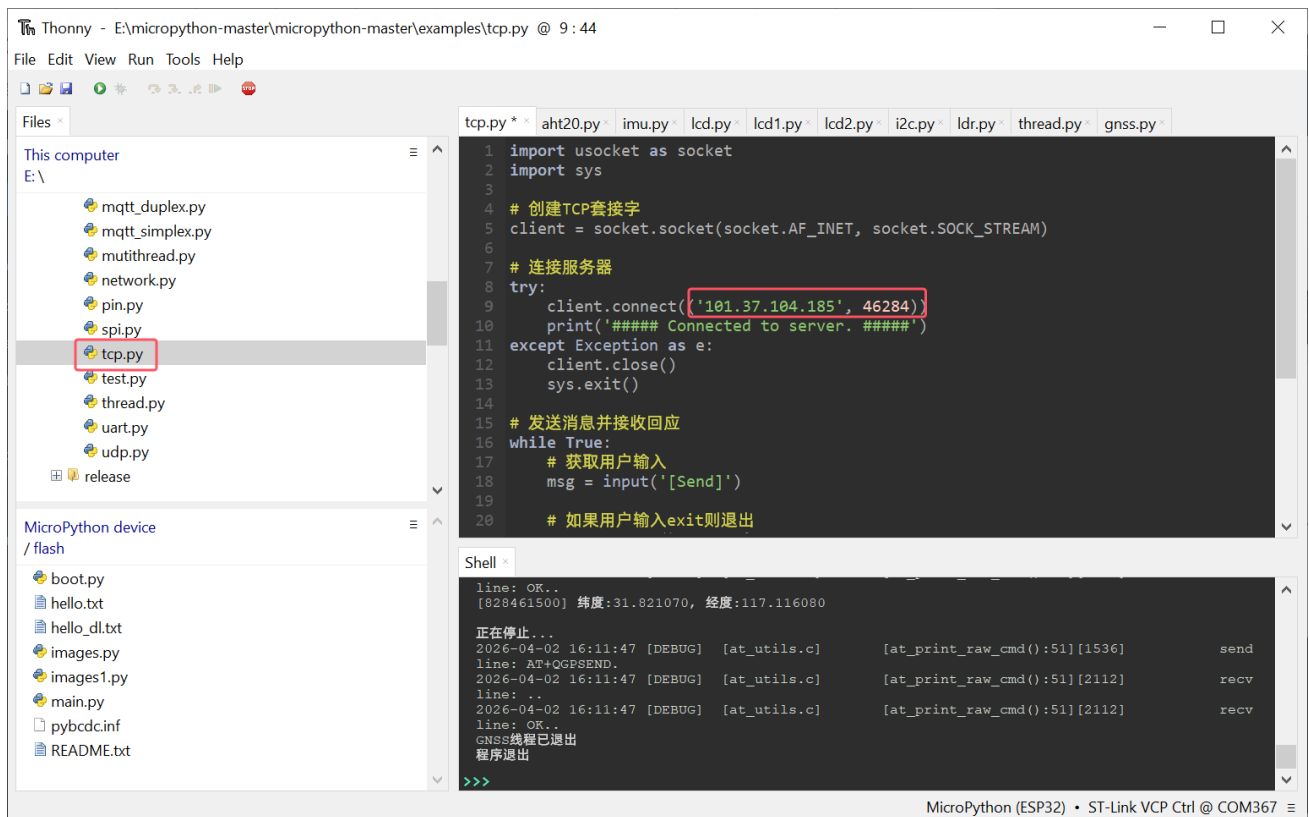


图 24: TCP 服务器地址与端口配置

点击运行程序，即可建立 TCP 连接，之后发送数据，比如 test123

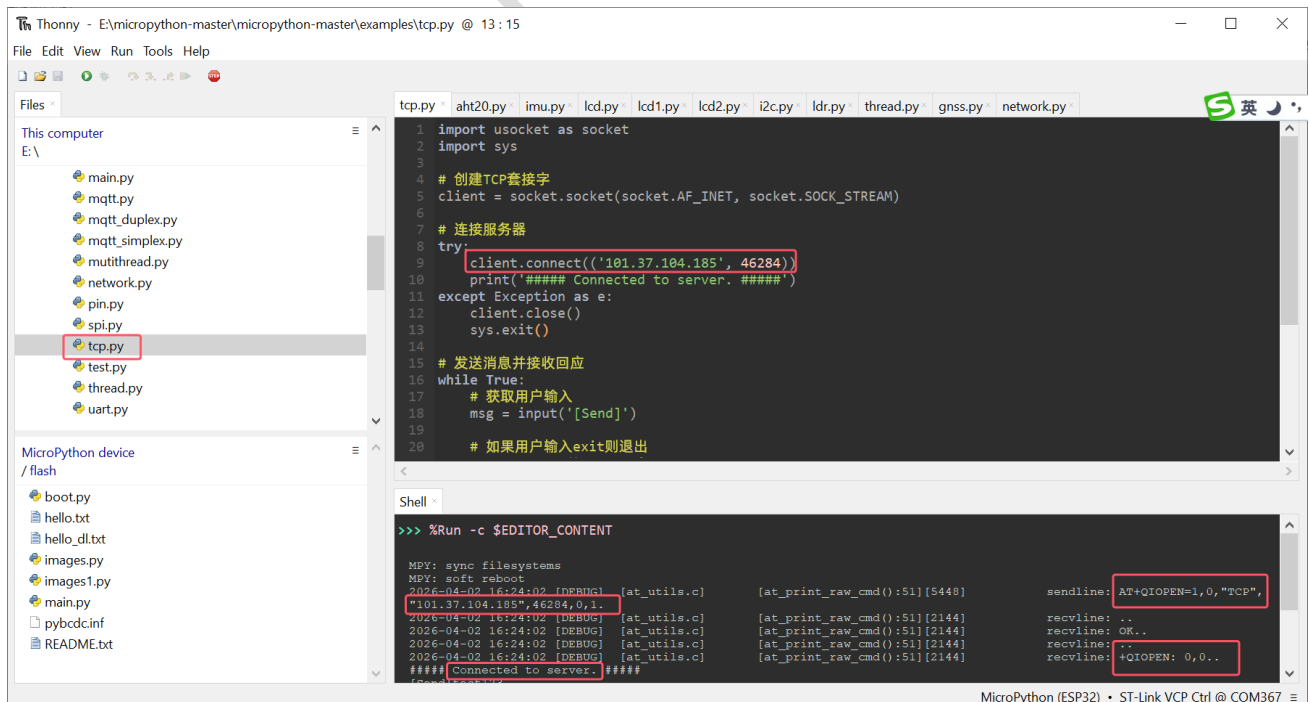


图 25: 建立 TCP 连接

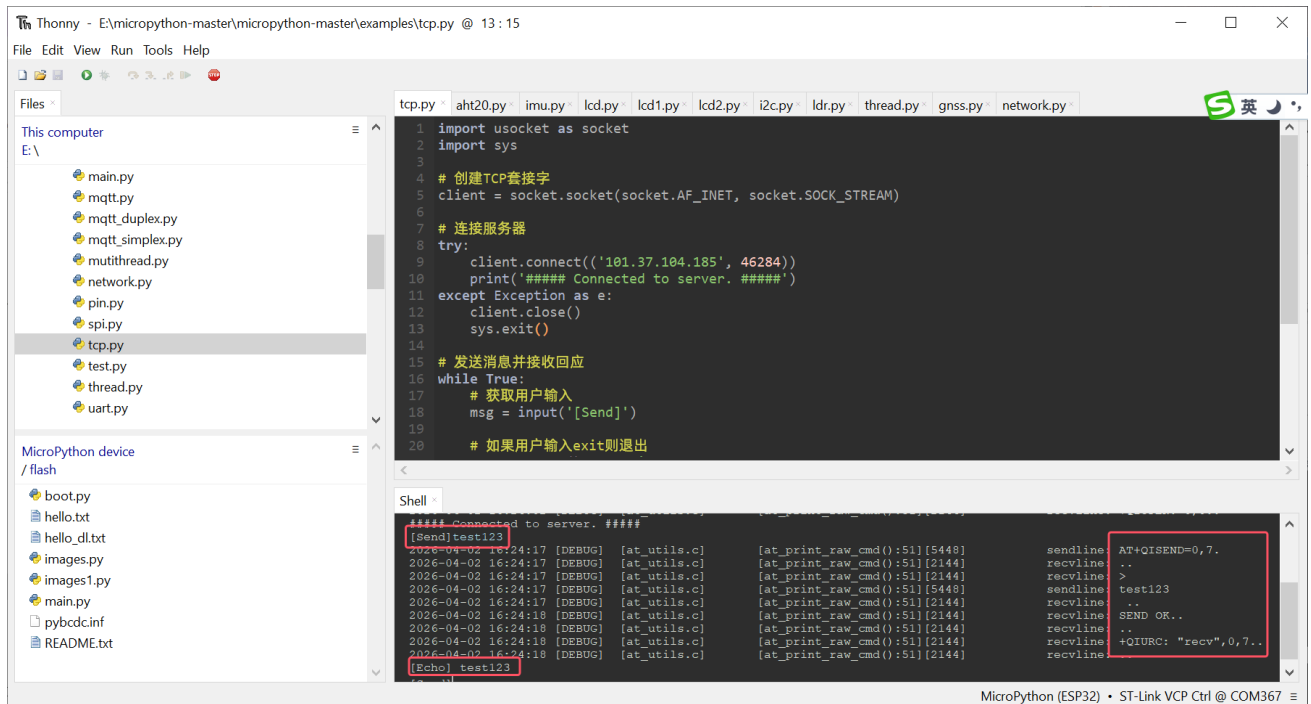


图 26: 发送 TCP 数据

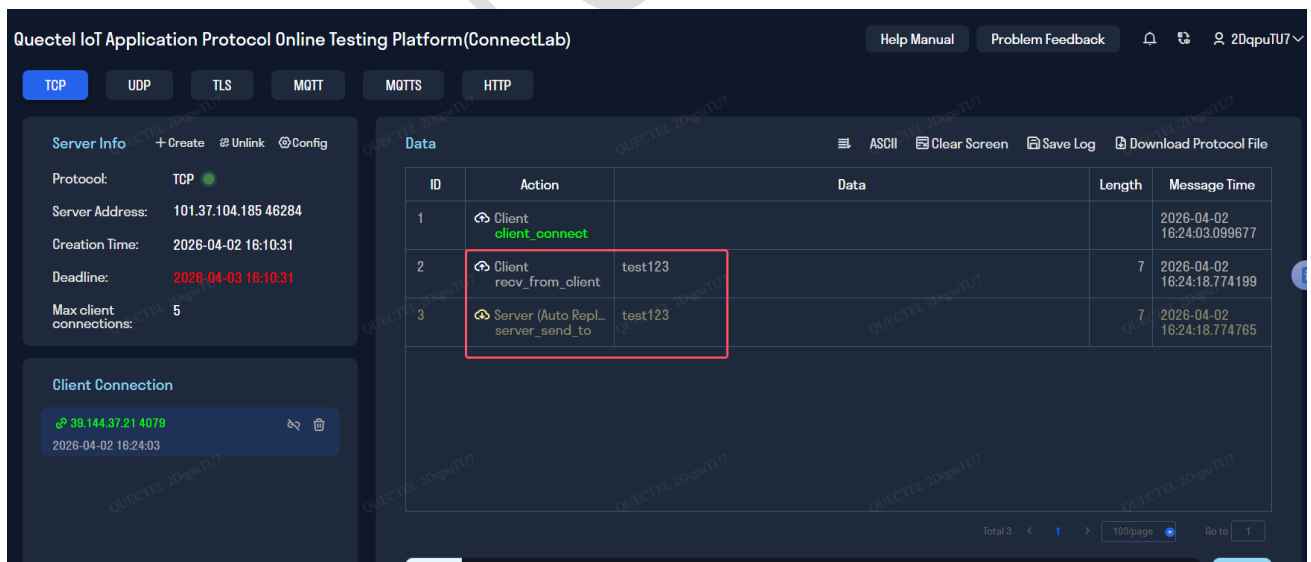


图 27: ConnectLab TCP 服务器接收数据

输入 exit, 用于关闭 TCP 连接

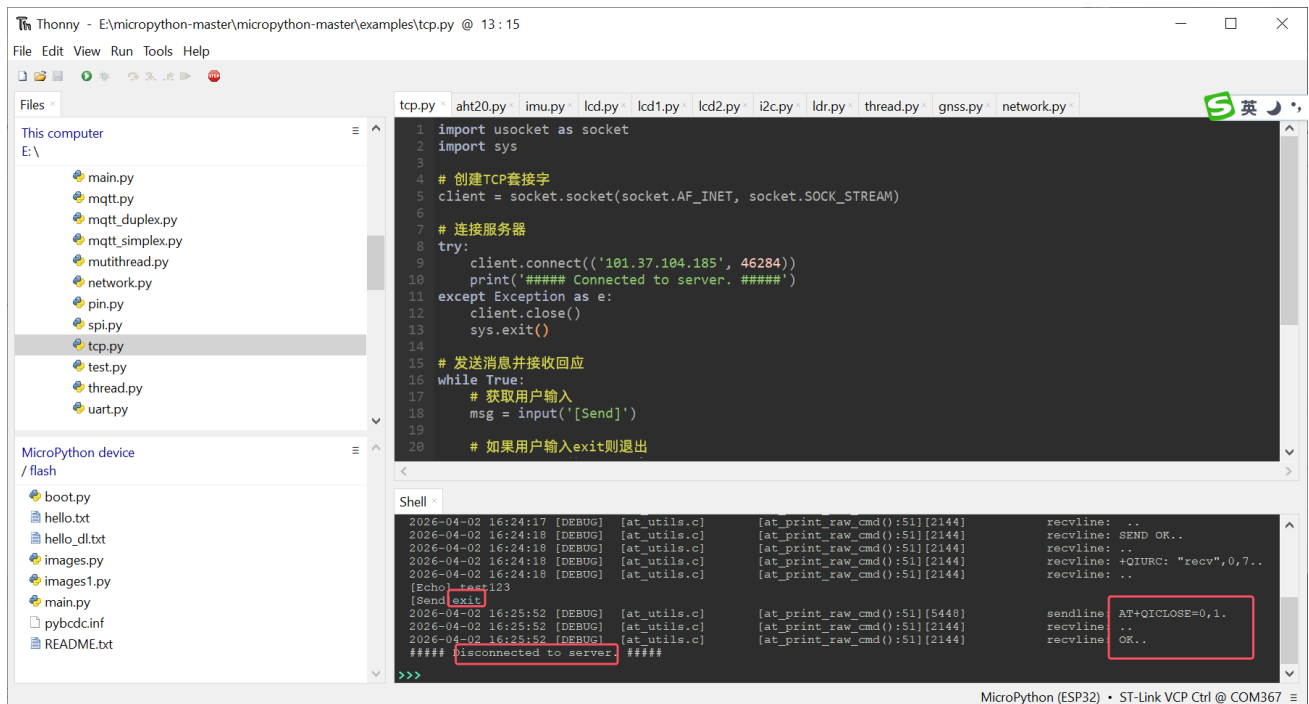


图 28: 关闭 TCP 连接

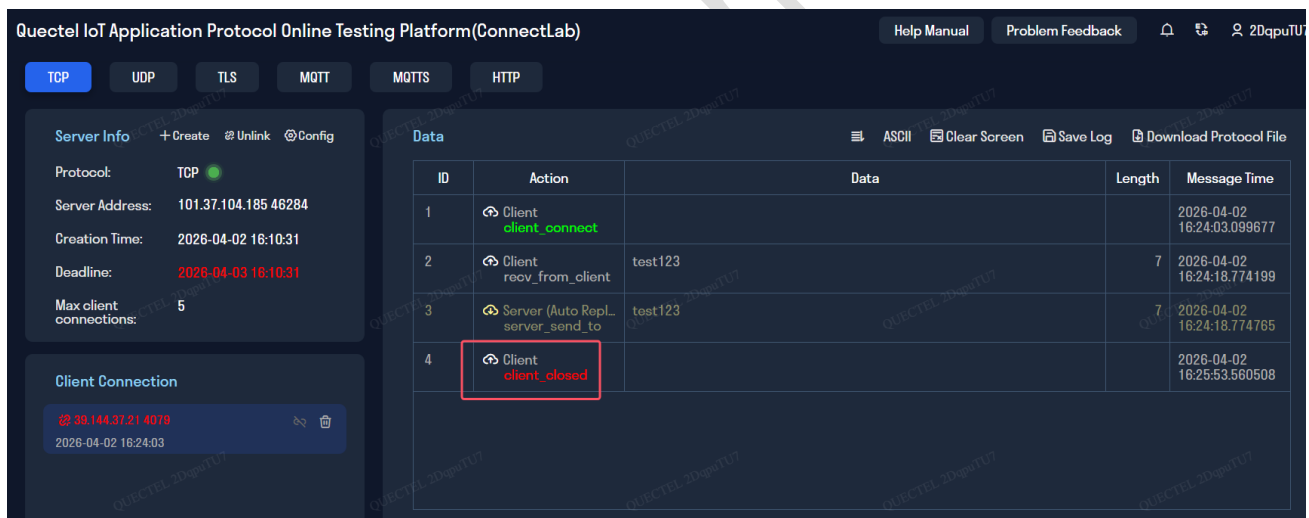


图 29: ConnectLab TCP 服务器运行结果

3.11. HTTP 客户端测试

http.py 用于 MQTT 测试，http_simplex.py 用于 HTTPS 单向认证测试，http_duplex.py 用于 HTTPS 双向认证测试。

下面举例说明 HTTP 测试。

首先在 connectlab 上打开 HTTP 服务器，如下图所示。

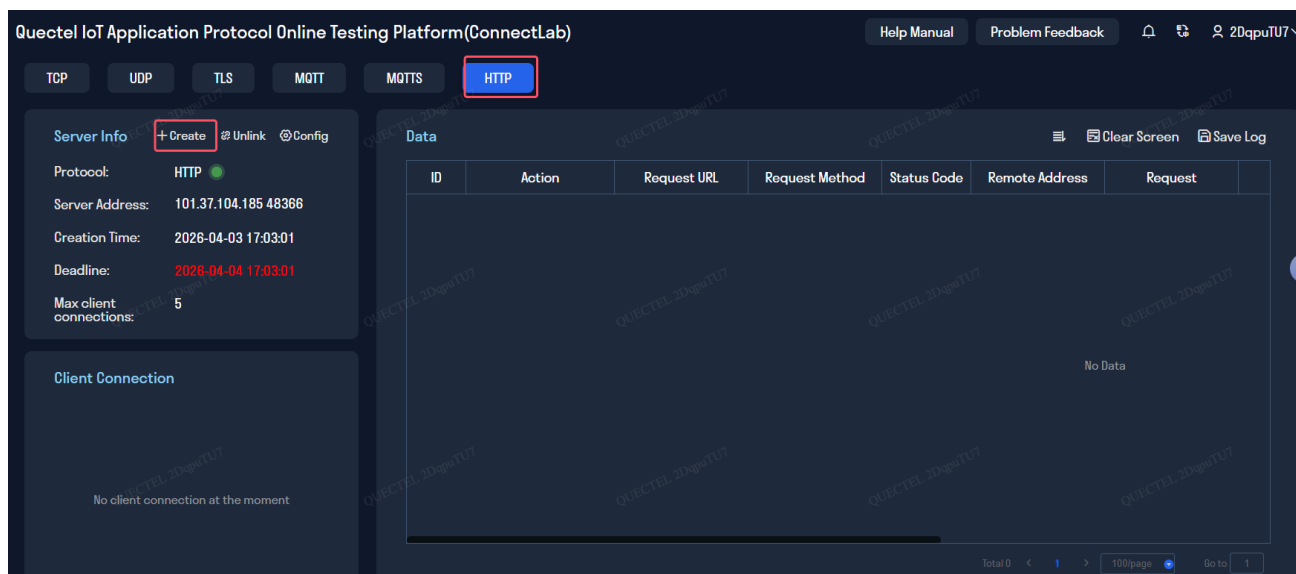


图 30: ConnectLab HTTP 服务器

点击“配置”，配置自定义响应，注意 path、method、params、response_dat 的配置。

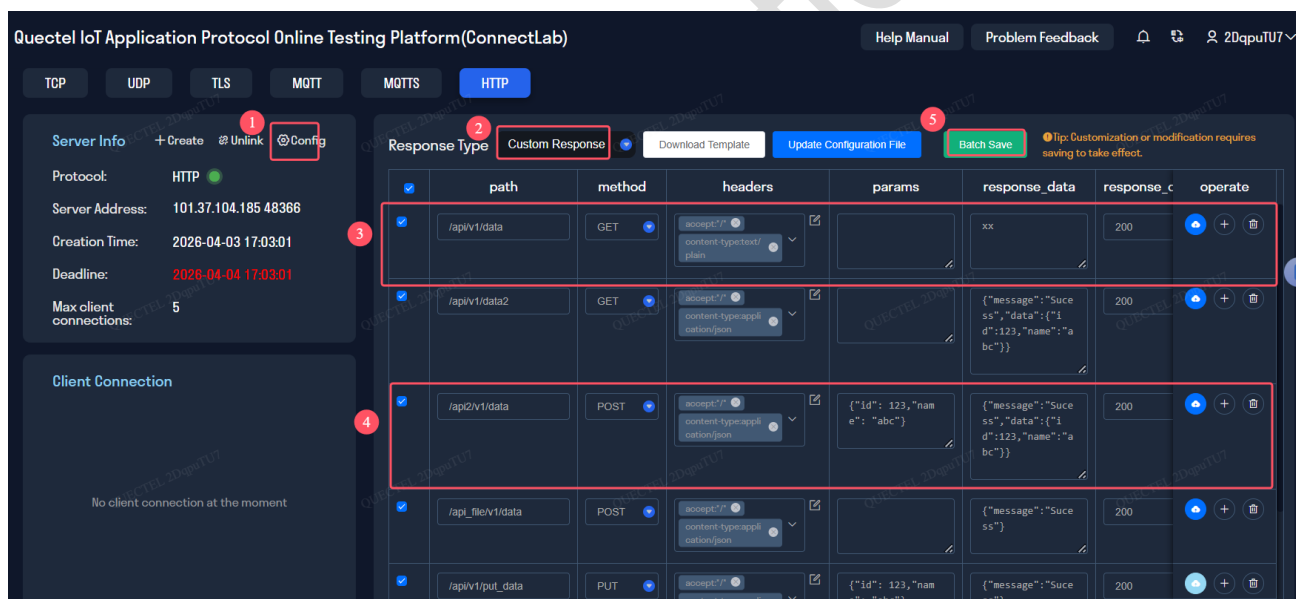


图 31: ConnectLab HTTP 服务器配置

http.py 代码里配置 HTTP GET URL 与 POST URL 及 POST data.

```

1 import urequests
2 import json
3
4 def http_get_test():
5     # get_url = "http://112.31.84.164:8300/1024.txt"
6     get_url = 'http://101.37.104.185:48366/api/v1/data'
7     print(f'### GET URL: {get_url}')
8     print('### Starting GET request...\n')
9     try:
10         get_response = urequests.get(get_url)
11         print(f'\n### GET successfully !')
12         print(f'### Status Code: {get_response.status_code}\n')
13
14         if get_response.status_code == 200:
15             print(f'### Response Content: {get_response.text}')
16         else:
17             print(f'*** GET failed.')
18
19         get_response.close()
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

```

2026-04-03 17:04:11 [DEBUG] [at_utils.c] [at_print_raw_cmd():51][2112] recvline: ..
2026-04-03 17:04:11 [DEBUG] [at_utils.c] [at_print_raw_cmd():51][2112] recvline: OK..
### Response Content: {
  "message": "Success",
  "data": {
    "id": 123,
    "name": "abc"
  }
}
=====
### Test done.
>>>

```

图 32: HTTP GET 配置

```

20
21 except Exception as e:
22     print(f'\n*** GET request exception. Err: {e}')
23
24
25 def http_post_test():
26     # post_url = "http://112.31.84.164:8300/upload.php"
27     post_url = 'http://101.37.104.185:48366/api2/v1/data'
28     post_data = {'id': 123, 'name': 'abc'}
29     post_header = {'Content-Type': 'application/json'}
30
31     print(f'\nPOST URL: {post_url}')
32     print('### Starting POST request...\n')
33     try:
34         post_response = urequests.post(
35             post_url,
36             data = json.dumps(post_data),
37             headers = post_header
38         )
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

```

2026-04-03 17:04:11 [DEBUG] [at_utils.c] [at_print_raw_cmd():51][2112] recvline: ..
2026-04-03 17:04:11 [DEBUG] [at_utils.c] [at_print_raw_cmd():51][2112] recvline: OK..
### Response Content: {
  "message": "Success",
  "data": {
    "id": 123,
    "name": "abc"
  }
}
=====
### Test done.
>>>

```

图 33: HTTP POST 配置

查看成功运行结果：

```

Shell x
2026-04-03 17:08:31 [DEBUG] [at_utils.c] [at_print_raw_cmd():51][4616] sendline: ..
2026-04-03 17:08:31 [DEBUG] [at_utils.c] [at_print_raw_cmd():51][2112] recvline: ..
2026-04-03 17:08:31 [DEBUG] [at_utils.c] [at_print_raw_cmd():51][2112] recvline: SEND OK..
2026-04-03 17:08:31 [DEBUG] [at_utils.c] [at_print_raw_cmd():51][2112] recvline: ..
2026-04-03 17:08:31 [DEBUG] [at_utils.c] [at_print_raw_cmd():51][2112] recvline: +QIURC: "recv",0,167..
2026-04-03 17:08:31 [DEBUG] [at_utils.c] [at_print_raw_cmd():51][2112] recvline: ..
2026-04-03 17:08:31 [DEBUG] [at_utils.c] [at_print_raw_cmd():51][2112] recvline: ..
2026-04-03 17:08:31 [DEBUG] [at_utils.c] [at_print_raw_cmd():51][2112] recvline: +QIURC: "closed",0..

### GET successfully !
### Status Code: 200
    
```

图 34: HTTP GET 运行结果

```

Shell x
2026-04-03 17:08:34 [DEBUG] [at_utils.c] [at_print_raw_cmd():51][2112] recvline: SEND OK..
2026-04-03 17:08:34 [DEBUG] [at_utils.c] [at_print_raw_cmd():51][2112] recvline: ..
2026-04-03 17:08:34 [DEBUG] [at_utils.c] [at_print_raw_cmd():51][2112] recvline: +QIURC: "recv",0,247..
2026-04-03 17:08:34 [DEBUG] [at_utils.c] [at_print_raw_cmd():51][2112] recvline: ..
2026-04-03 17:08:34 [DEBUG] [at_utils.c] [at_print_raw_cmd():51][2112] recvline: ..
2026-04-03 17:08:34 [DEBUG] [at_utils.c] [at_print_raw_cmd():51][2112] recvline: +QIURC: "closed",0..

### POST successfully !
### Status Code: 200

2026-04-03 17:08:34 [DEBUG] [at_utils.c] [at_print_raw_cmd():51][4592] sendline: AT+QICLOSE=0,1.
2026-04-03 17:08:34 [DEBUG] [at_utils.c] [at_print_raw_cmd():51][2112] recvline: ..
2026-04-03 17:08:35 [DEBUG] [at_utils.c] [at_print_raw_cmd():51][2112] recvline: OK
    
```

图 35: HTTP POST 运行结果

Quectel IoT Application Protocol Online Testing Platform(ConnectLab)

Help Manual Problem Feedback 20qpuTUT

TCP UDP TLS MQTT MQTTS HTTP

Server Info + Create @ Unlink @ Config

Protocol: HTTP

Server Address: 101.37.104.185 48386

Creation Time: 2026-04-03 17:03:01

Deadline: 2026-04-04 17:03:01

Max client connections: 5

Client Connection

39.144.37.124 2026-04-03 17:08:31

Data

ID	Action	Request URL	Request Method	Status Code	Remote Address	Request
1	Client_Server request_respon...	http://101.37.104.185/api/v1/data	GET	200	101.37.104.185	{ "method": "GET", "scheme": "http", "host": "101.37.104.185", "request_url": "http://101.37.104.185/api/v1/data", "raw_path": "/api/v1/data", "path": "/api/v1/data" }
2	Client_Server request_respon...	http://101.37.104.185/api/v1/data	POST	200	101.37.104.185	{ "method": "POST", "scheme": "http", "host": "101.37.104.185", "request_url": "http://101.37.104.185/api/v1/data", "raw_path": "/api/v1/data", "path": "/api/v1/data" }

Clear Screen Save Log Download Protocol File

图 36: ConnectLab HTTP 服务器运行结果

3.12. MQTTS 客户端测试

mqtt.py 用于 MQTT 测试, mqtt_simplex.py 用于 MQTTS 单向认证测试, mqtt_duplex.py 用于 MQTTS 双向认证测试。

下面举例说明 MQTTS 双向认证。

首先在 connectlab 上打开 MQTTS 双向认证服务器, 如下图所示。

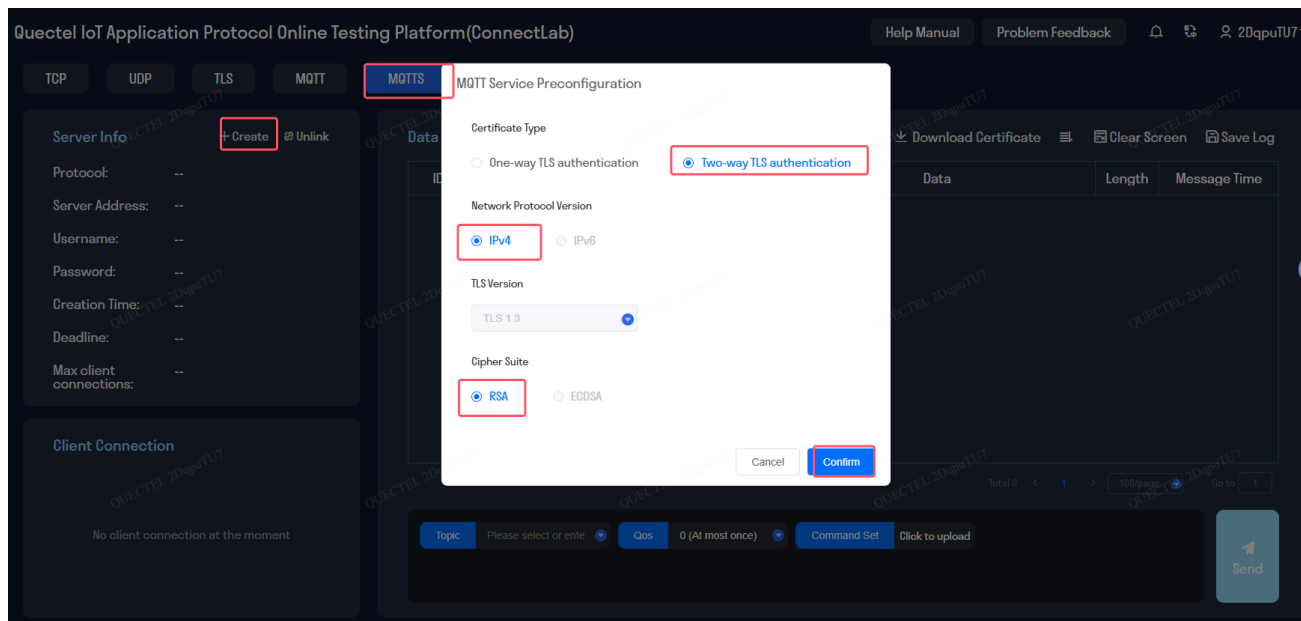


图 37: ConnectLab MQTTS 服务器

从 connectlab 下载 MQTTS 客户端所需证书, 如下图所示。

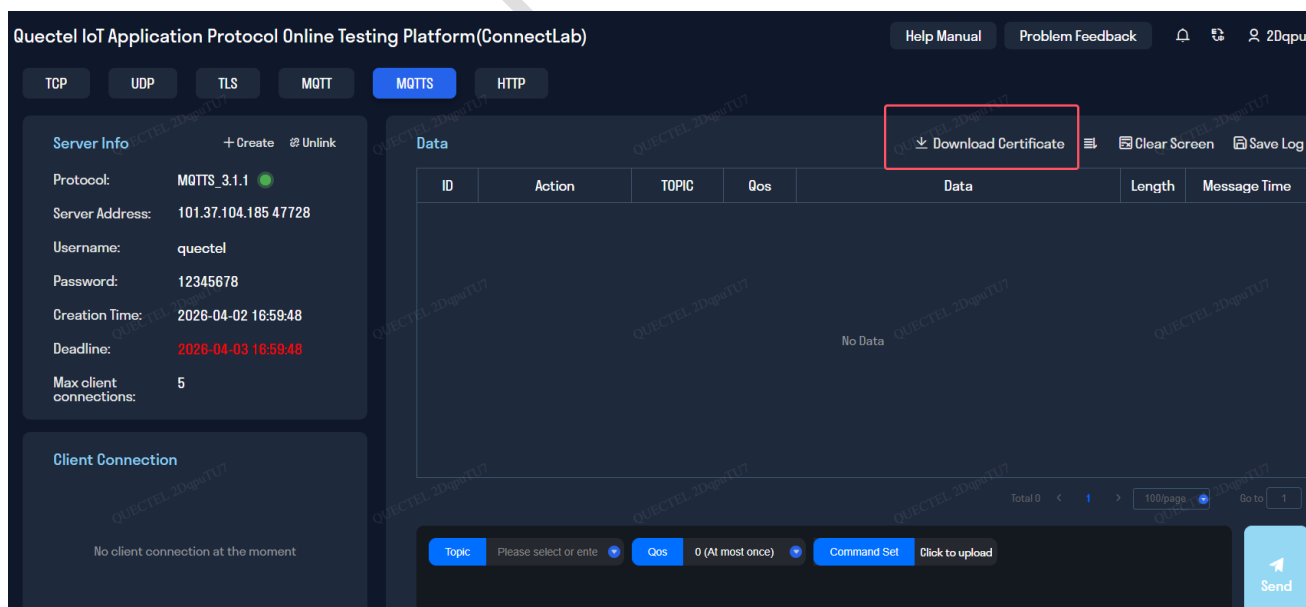


图 38: ConnectLab MQTTS 服务器证书下载

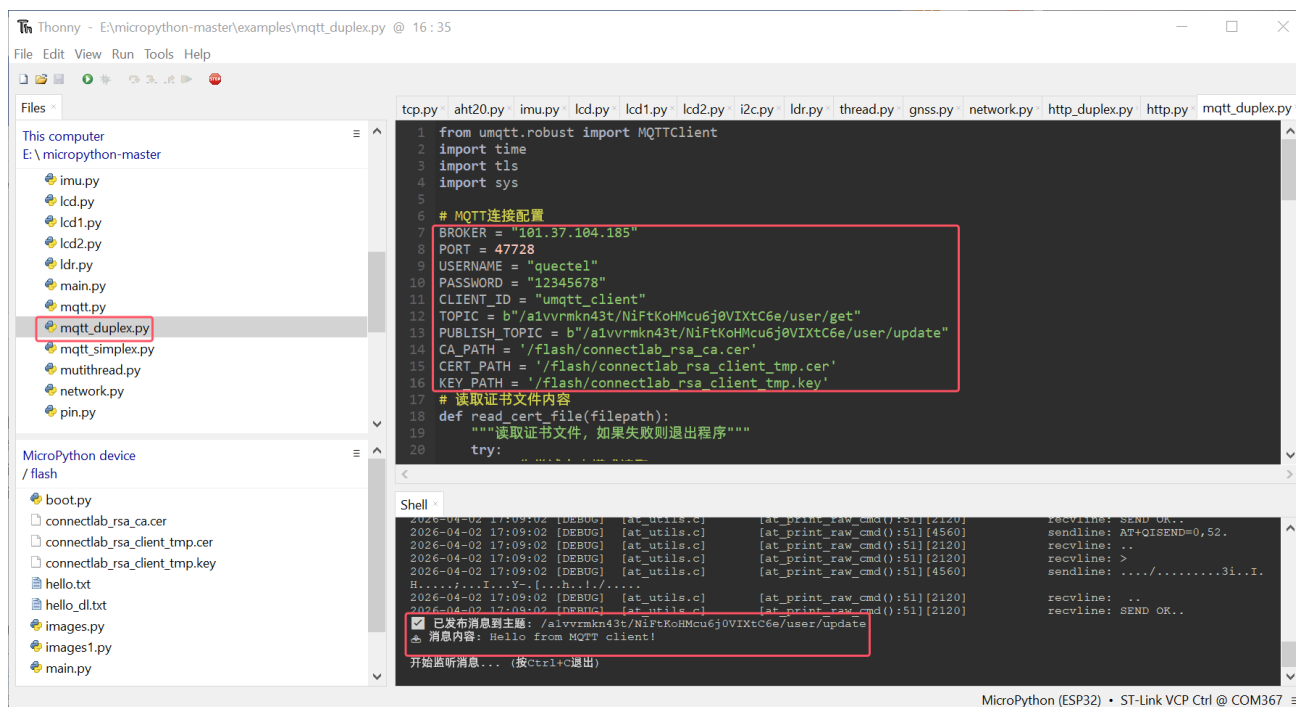


图 40: MQTTS 配置及运行结果

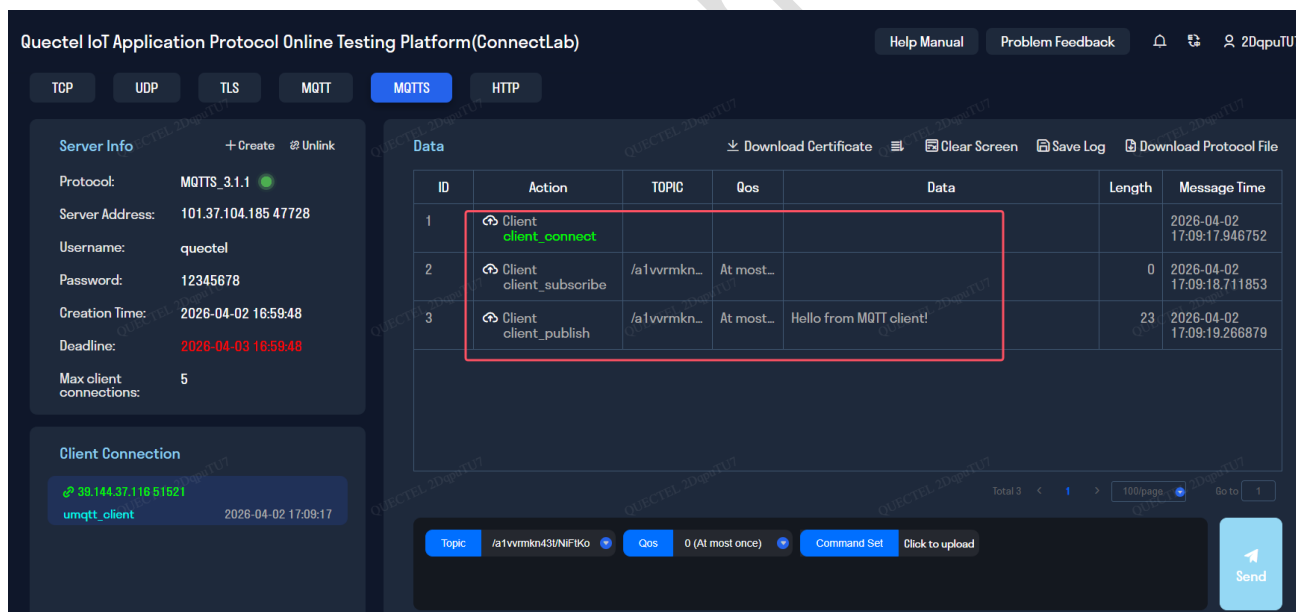


图 41: ConnectLab MQTTS 服务器运行结果

3.13. FTP 客户端测试

ftp2.py 用于 FTP 测试，运行结果如下。ftps.py 用于 FTPS 测试。

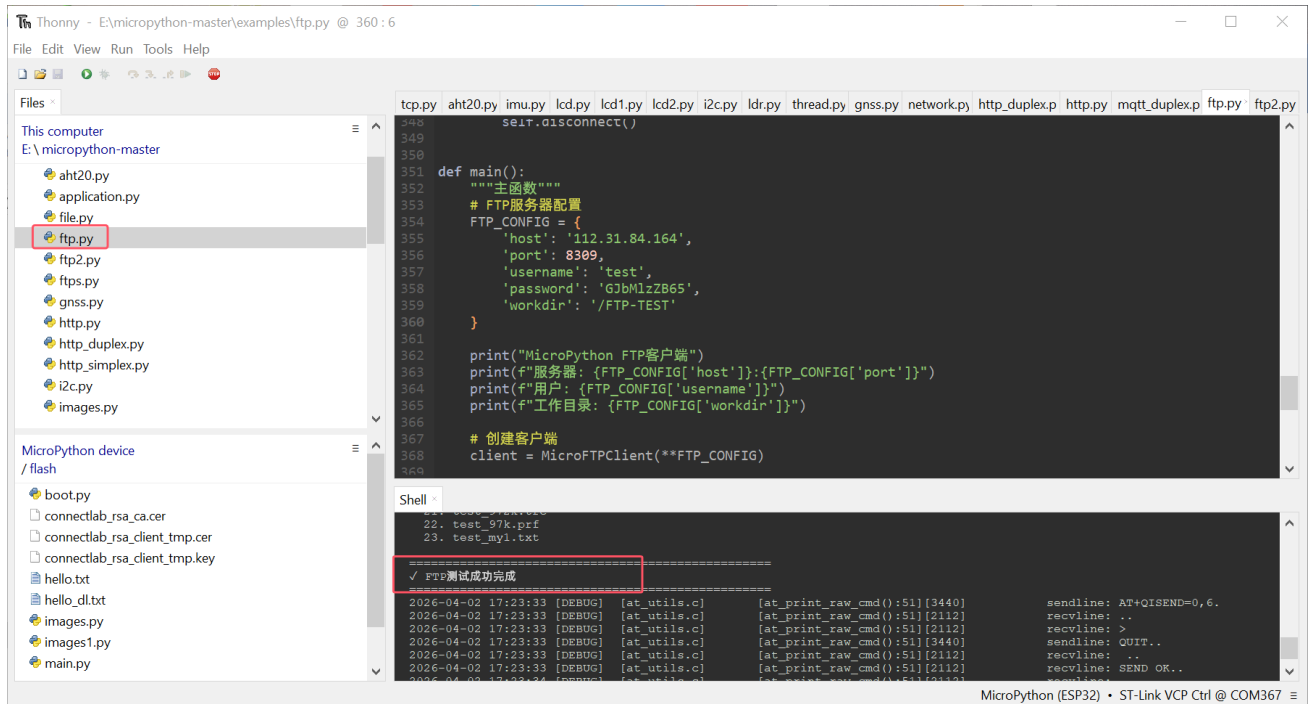


图 42: ConnectLab FTP 服务器运行结果

4 参考文档

表 2: 参考文档

序号	Document Name 文档名称
[1]	Quectel_UniKnect_Gen1_Pro_硬件指导
[2]	Quectel_UniKnect_Gen1_Pro_软件开发指导
[3]	LCD-API 参考手册
[4]	外设 API 参考手册
[5]	Thread-API 参考手册
[6]	Network&GNSS&File-API 参考手册
[7]	TCP&UDP 客户端 API 参考手册
[8]	HTTP 客户端 API 参考手册
[9]	MQTT 客户端 API 参考手册
[10]	FTP 客户端 API 参考手册